含 AI 生成

《Web 编程技术》期末大作业

智问 · AI 智能问答平台

——网站设计报告——

课程名称： Web 编程技术

设计题目： 智问 · AI 智能问答平台

学　号： 20232653

班　级： 物联网 2301

姓　名： 庞志达

完成日期： 2026 年 10 月

目　　录

第一章 绪论

1.1 项目背景与意义

1.2 国内外研究与应用现状

1.3 设计目标

1.4 主要工作与报告结构

第二章 需求分析

2.1 用户角色分析

2.2 功能性需求

2.3 非功能性需求

2.4 用例描述

第三章 系统总体设计

3.1 系统架构设计

3.2 技术选型与理由

3.3 分层设计

3.4 工程目录结构

3.5 核心业务流程

第四章 数据库设计

4.1 概念结构设计

4.2 逻辑结构设计

4.3 数据表详细设计

4.4 索引设计与优化说明

4.5 初始化数据说明

第五章 系统详细设计与实现

5.1 统一响应与全局异常处理

5.2 用户认证与权限控制

5.3 问答引擎（核心算法）

5.4 会话与问答历史

5.5 知识库与分类管理

5.6 用户管理与系统配置

5.7 统计看板实现

第六章 前端设计与实现

6.1 页面结构与样式体系

6.2 响应式与交互实现

6.3 请求封装与鉴权处理

6.4 图表渲染

第七章 系统测试

7.1 测试环境与方法

7.2 功能测试用例

7.3 接口测试

7.4 性能与兼容性测试

7.5 测试结论

第八章 系统部署与运行

8.1 环境准备

8.2 数据库部署

8.3 配置修改与启动

8.4 常见问题排查

第九章 总结与展望

9.1 完成的工作

9.2 存在的不足

9.3 后续改进方向

参考文献

附录 A 接口清单

附录 B 工程文件清单

第一章 绪论

1.1 项目背景与意义

随着移动互联网与人工智能技术的快速普及，用户获取信息的方式正在从「逐页浏览」转向「直接提问」。传统的校园信息发布网站通常把通知、办事指南、常见问题分散在不同的栏目与页面中，用户需要反复点击、层层翻找才能定位到想要的内容，检索效率低、体验差。而基于问答形式的信息获取方式，允许用户用自然语言描述自己的困惑，系统直接返回与之匹配的答案，显著降低了信息获取的门槛。

本项目「智问 · AI 智能问答平台」正是在这一背景下设计并实现的。它把常见问题整理为结构化的知识库条目，通过相似度算法把用户问题与标准问题、关键词进行匹配打分，命中后直接返回可追溯的标准答案；同时预留了大模型接入能力，管理员可以在后台一键把回答方式切换为调用 OpenAI 兼容接口的大模型生成，并在大模型调用失败时自动降级回本地知识库检索。

从课程学习的角度看，本项目是一次对 Java Web 开发全流程的完整实践。它覆盖了需求分析、数据库设计、后端分层开发、前端页面与交互实现、接口联调、系统测试与部署等环节，涉及 Spring Boot、MyBatis-Plus、MySQL、Thymeleaf、JWT 等主流技术，能够较为全面地检验并巩固《Web 编程技术》课程所学的知识点。同时，项目选择了「知识库检索 + 大模型生成」这一具有现实意义的主题，使技术练习与真实应用场景相结合，具有较强的实践意义。

1.2 国内外研究与应用现状

问答系统按照实现方式大致可以分为三类：基于规则与模板的问答、基于检索的问答以及基于生成式大模型的问答。

基于规则与模板的问答系统出现最早，通常依靠人工编写的关键词规则与句式模板来匹配问题，优点是实现简单、结果可控，缺点是规则维护成本高、泛化能力弱，稍有变化的问法就可能无法识别。

基于检索的问答把常见问题与标准答案整理成问答对，用户提问时通过文本相似度算法在库中检索最接近的条目。这类系统在工业界应用非常广泛，例

如各类智能客服、政务问答、校园办事指南等场景。它的核心在于相似度计算，常见做法包括基于词频的 TF-IDF、向量空间模型、BM25，以及基于词向量的语义相似度。本项目的本地检索策略即属于这一类，考虑到中文分词需要引入外部词典、部署成本较高，项目采用字符二元组加杰卡德相似度作为文本表示与度量方式，实现简单且无需外部依赖。

生成式问答随着大语言模型的成熟迅速兴起，代表产品包括 ChatGPT 等。这类系统能够生成流畅自然、覆盖面广的回答，但在垂直领域存在「幻觉」风险，且依赖外部网络与密钥，响应成本较高。因此，在校园、政务等对答案准确性要求较高的场景中，「检索 + 生成」的混合架构逐渐成为主流：优先用可控的知识库兜底，再用大模型提升回答的灵活性。本项目正是采用了这一思路，把两种策略解耦为可插拔的组件，并实现了失败降级。

从技术生态看，Spring Boot 已成为 Java Web 后端开发事实上的标准框架，MyBatis-Plus 在 MyBatis 之上提供了大量开箱即用的能力（分页插件、逻辑删除、公共字段自动填充等），极大地提升了开发效率。前后端分离与「服务端渲染骨架 + 接口异步取数」两种模式在实践中各有应用，本项目前台采用 Thymeleaf 渲染页面骨架、再由原生 JavaScript 通过 fetch 调用 REST 接口取数的方式，兼顾了页面首屏速度与交互灵活性。

1.3 设计目标

本项目希望达成以下设计目标：

（1）功能完整：前台提供注册、登录、智能问答、问答历史与答案反馈；后台提供数据看板、知识库管理、分类管理、用户管理、问答记录管理与系统参数配置，形成从用户侧到管理侧的完整闭环。

（2）问答可用：默认情况下不依赖任何外部服务即可离线回答常见问题，命中知识库时给出可追溯的标准答案与相似度得分；同时支持切换到大模型模式，并在其失败时自动降级。

（3）结构清晰：后端严格分层（Controller / Service / Mapper / Entity），统一响应体、统一异常处理、统一鉴权，代码具备良好的可读性与可扩展性。

（4）安全可靠：密码采用随机盐加 SHA-256 存储，接口通过 JWT 鉴权，管理端接口进行角色校验，参数进行合法性校验，防止越权与非法数据进入系统。

（5）体验良好：页面采用响应式布局，问答交互顺畅，后台看板用图表直观呈现运营数据。

（6）性能可观测：通过连接池、分页上限、内存快照缓存、SQL 原子自增、统计下推数据库等手段，保证系统在数据量增长时仍能保持稳定响应。

1.4 主要工作与报告结构

本项目的主要工作包括：需求分析与用例梳理；数据库概念结构与逻辑结构设计，完成 6 张数据表的建表与索引设计；基于 Spring Boot 3 与 MyBatis-Plus 完成后端分层开发，实现统一响应、全局异常处理、JWT 鉴权与权限拦截；设计并实现问答引擎，包含本地检索策略与大模型策略以及失败降级机制；完成前台 5 个页面与后台 7 个页面的开发与联调；编写初始化数据脚本与部署说明；并对系统进行功能、接口、性能与兼容性测试。

本报告共分九章。第一章为绪论，介绍项目背景、现状与目标；第二章为需求分析；第三章为系统总体设计，给出架构图、技术选型与核心流程；第四章为数据库设计；第五章为系统详细设计与实现，重点阐述问答引擎等核心模块；第六章为前端设计与实现；第七章为系统测试；第八章为部署与运行说明；第九章为总结与展望。报告最后附有参考文献、接口清单与工程文件清单。

第二章 需求分析

2.1 用户角色分析

系统面向两类用户角色：普通用户与管理员。两类角色共用同一套用户表，通过 role 字段区分，登录后由 JWT 中携带的角色信息决定可访问的页面与接口。

角色	标识	主要职责	可访问范围
普通用户	USER	注册登录、提出问题、查看与管理自己的问答历史、对答案进行反馈	前台首页、登录/注册、智能问答、问答历史，以及 /api/chat/** 等需登录接口
管理员	ADMIN	在后台维护知识库与分类、管理用户、查看问答记录与运营数据、调整问答引擎与站点参数	在普通用户权限基础上，额外可访问 /admin/** 页面与 /api/admin/** 接口

表 2-1 用户角色与权限划分

2.2 功能性需求

功能性需求按前台与后台两条主线梳理。前台面向普通用户，强调「问得到、看得到、管得住」；后台面向管理员，强调「数据可视、内容可控、参数可调」。

2.2.1 前台功能需求

编号	功能名称	需求描述
F-01	首页展示	展示站点名称与首页公告（来自系统参数）、平台规模数据（知识条目数、注册用户数、知识库命中率）以及热门问题列表，未登录用户亦可浏览。
F-02	用户注册	填写用户名、昵称、邮箱（选填）、密码与确认密码完成注册；用户名需为 4~20 位字母、数字或下划线，密码长度 6~32 位，两次密码需一致，用户名不可重复。
F-03	用户登录	使用用户名与密码登录，登录成功后签发 JWT 并写入 Cookie，页面跳转与接口调用共用凭证。
F-04	智能问答	选择或新建会话后输入问题（最长 500 字），系统返回答案，并展示答案来源、相似度得分、引擎耗时与命中的标准问题（依据）。
F-05	猜你想问	在问答页展示按热度排序的推荐问题，点击后自动填充并提问。
F-06	答案反馈	对系统回答点击「有帮助」或「没帮助」，反馈结果写入消息记录用于运营分析。
F-07	会话管理	左侧展示我的会话列表，可新建会话、切换会话、删除单个会话、清空全部会话。
F-08	问答历史	分页展示我的全部会话，可展开查看完整对话内容，可删除或清空。

F-09	个人资料	查看当前登录用户信息，修改昵称与邮箱，修改登录密码（需校验原密码）。
F-10	退出登录	清除本地与服务端 Cookie 凭证，返回登录页。

表 2-2 前台功能性需求

2.2.2 后台功能需求

编号	功能名称	需求描述
B-01	管理员登录	使用管理员账号登录后台，非 ADMIN 角色账号登录时被拒绝。
B-02	数据看板	展示注册用户数、问答会话数、提问总数、知识条目数、今日提问数、知识库命中率、答案满意度 7 个统计卡，以及近 7 天提问趋势、答案来源占比、分类知识量、24 小时活跃分布 4 张图表与热门知识 TOP10、用户提问 TOP10 两张榜单。
B-03	知识库管理	分页展示知识条目，支持按关键词、分类、状态筛选，支持新增、编辑、删除与启用/停用。
B-04	分类管理	展示分类及其下知识条目数量，支持新增、编辑与删除；分类下仍有条目时禁止删除。
B-05	用户管理	分页展示用户，支持按关键词、角色筛选，支持启用/禁用、重置密码与删除；不允许禁用或删除当前登录的管理员自身。
B-06	问答记录管理	分页展示全部会话，支持按标题搜索、查看对话详情与删除会话。
B-07	系统配置	查看与修改问答模式、命中阈值、大模型接口地址/密钥/模型名，以及站点名称与首页公告，修改后无需重启服务即可生效。

表 2-3 后台功能性需求

2.3 非功能性需求

2.3.1 性能需求

（1）响应时间：在本地开发环境、单机 MySQL 与默认演示数据规模下，问答接口的引擎耗时应在几十毫秒量级，普通列表与统计接口的响应时间应在数百毫秒以内。

（2）并发能力：通过 HikariCP 连接池（最大连接数 20）与 Tomcat 线程池（最大线程数 200）支撑一定并发；问答接口对知识库采用 60 秒内存快照，避免每轮问答触发全表扫描。

（3）可扩展性：数据量增长后，列表接口通过分页（单页上限 100 条）与索引控制查询成本；统计接口尽量下推到数据库用 GROUP BY 完成，避免把明细数据加载到内存。

2.3.2 安全需求

（1）密码安全：不明文存储密码，采用随机盐加 SHA-256 摘要，存储格式为「盐值:哈希值」，抵御彩虹表攻击。

（2）身份认证：接口通过 JWT 认证，token 有效期 7200 秒，签发时携带用户 ID、用户名与角色。

（3）访问控制：拦截器对 /api/**、/admin/**、/chat、/history 等路径统一鉴权，管理端路径额外校验 ADMIN 角色；未登录访问接口返回 401，越权访问返回 403。

（4）输入校验：注册、登录、提问、知识条目、分类等表单参数均使用 Jakarta Validation 注解校验，并在全局异常处理器中统一转换为友好提示。

（5）输出安全：密码字段使用 @JsonIgnore 避免随响应返回；前端对用户内容进行 HTML 转义，防止脚本注入。

2.3.3 易用性与可维护性需求

（1）界面简洁统一：前台与后台分别使用统一的样式表，颜色、圆角、阴影等通过 CSS 变量集中管理。

（2）响应式适配：页面在桌面、平板与手机浏览器上均可正常显示，手机端自动收起侧边栏。

（3）提示友好：操作成功/失败通过轻提示（toast）反馈，表单错误内联展示，破坏性操作二次确认。

（4）可维护性：后端严格分层，统一响应与异常处理，公共字段自动填充，减少重复代码；问答策略采用策略模式，新增策略无需修改引擎主体逻辑。

2.4 用例描述

系统主要用例包括：用户注册、用户登录、用户提问、查看问答历史、答案反馈、管理员登录、管理知识库、管理分类、管理用户、查看数据看板、修改系统配置。下面以「用户提问」这一核心用例为例给出详细描述，其余用例在表 2-4 中汇总。

项目	内容
用例名称	用户提问
参与者	普通用户（前置条件：已登录）
前置条件	用户已登录且账号状态正常
基本流程	1. 用户进入智能问答页；2. 选择已有会话或新建会话；3. 输入问题并发送；4. 系统落库用户提问；5. 问答引擎按配置策略求解；6. 返回答案并落库；7. 若命中知识条目则累加命中次数；8. 更新会话消息数与活跃时间；9. 前端展示答案与来源标签。
备选流程	a. 本地检索未命中（得分低于阈值）：返回兜底话术，来源标记为 fallback；b. 大模型模式且接口失败：自动降级到本地检索，来源标记为 knowledge；c. 会话不属于当前用户：返回 403 越权错误。
后置条件	qa_message 表新增两条记录（用户提问与系统回答），qa_session 表统计字段被更新

表 2-4 核心用例「用户提问」描述

用例	参与者	说明
用户注册	游客	校验用户名唯一性与两次密码一致性，密码加盐哈希后入库，默认角色 USER
用户登录	游客	校验用户名密码与账号状态，签发 JWT 并写入 Cookie
查看问答历史	普通用户	分页获取本人会话，展开查看会话内消息，可删除或清空
答案反馈	普通用户	对自己会话内的回答标记有帮助/没帮助
管理员登录	管理员	复用登录校验，额外要求角色为 ADMIN
管理知识库	管理员	新增/编辑/删除/启停知识条目，变更后使检索引擎内存快照失效
管理分类	管理员	新增/编辑/删除分类，分类下有条目时禁止删除
管理用户	管理员	启用/禁用、重置密码、删除用户，禁止操作自身账号
查看数据看板	管理员	聚合统计指标并渲染图表
修改系统配置	管理员	修改问答模式、阈值、大模型参数与站点信息，保存后刷新本地缓存

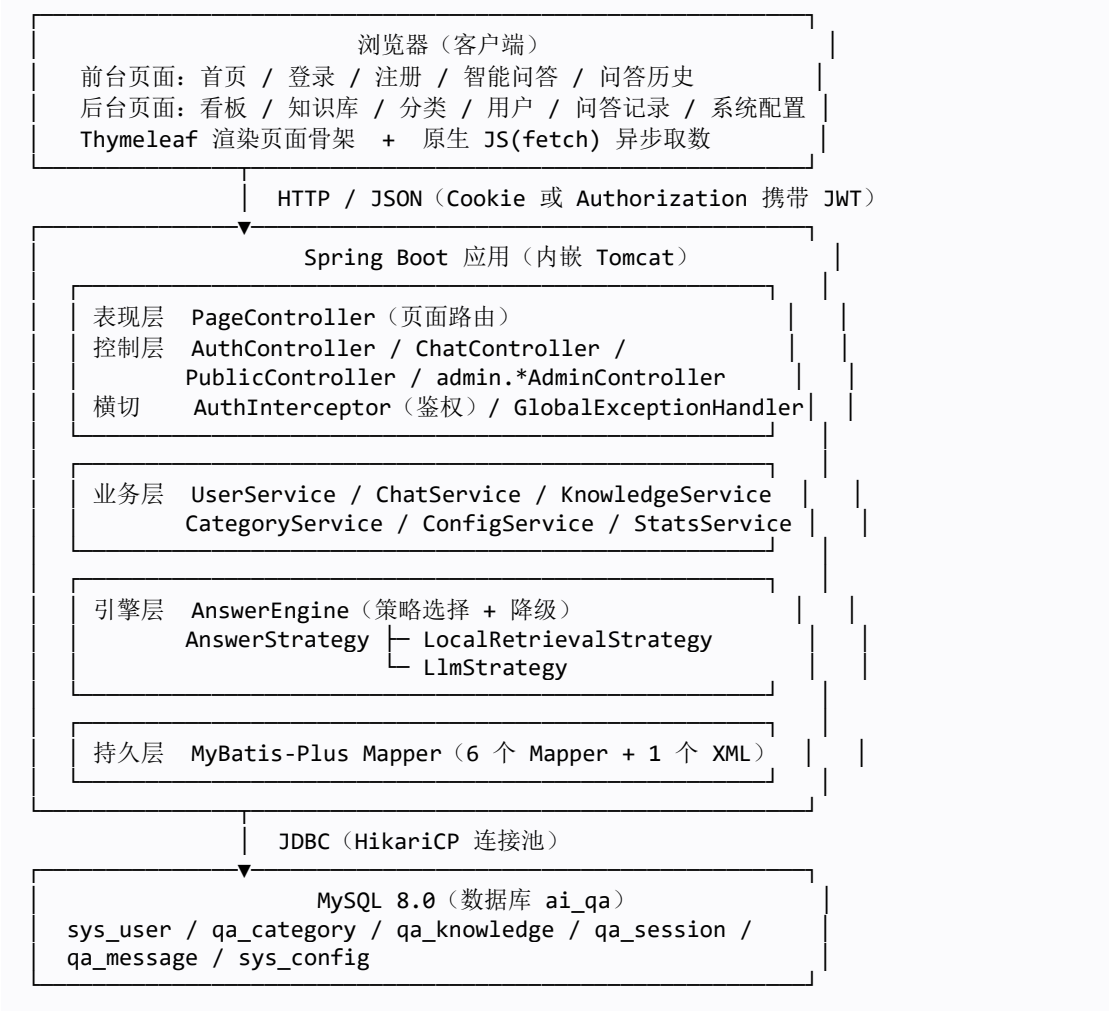
表 2-5 系统用例汇总

第三章 系统总体设计

3.1 系统架构设计

本系统采用 B/S（浏览器/服务器）架构，整体上属于典型的 Java Web 多层架构。浏览器负责页面渲染与交互，服务端由 Spring Boot 内嵌 Tomcat 承载，按照「表现层—控制层—业务层—引擎层—持久层—数据层」逐层向下，每一层只依赖其下层，职责单一、边界清晰。

图 3-1 系统总体架构图（示意）



一次典型的请求处理链路为：浏览器发起请求 → 内嵌 Tomcat 接收 → AuthInterceptor 进行登录与角色校验（校验通过后把当前用户写入 ThreadLocal 上下文 UserContext）→ Controller 接收并校验参数 → Service 处理业务（必要时调用 AnswerEngine 求解）→ Mapper 通过 MyBatis-Plus 访问数据库 → 结果封

装为统一响应体 Result 返回。若过程中抛出业务异常，则由 GlobalExceptionHandler 统一转换为标准响应，避免异常堆栈直接暴露给前端。

3.2 技术选型与理由

层次/用途	技术选型	版本	选择理由
运行环境	Java	17	Spring Boot 3 要求 Java 17 及以上；长期支持版本，语法特性完善
后端框架	Spring Boot	3.2.5	自动配置、内嵌容器、起步依赖，快速搭建可运行的 Web 应用
Web MVC	spring-boot-starter-web	随父版本	提供 REST 接口、参数绑定、内嵌 Tomcat
模板引擎	Thymeleaf	随父版本	服务端渲染页面骨架，与 Spring MVC 无缝集成，天然 HTML
持久层	MyBatis-Plus	3.5.7	在 MyBatis 基础上提供分页插件、逻辑删除、字段自动填充等能力
数据库	MySQL	8.0	开源稳定、生态成熟，utf8mb4 完整支持中文与表情符号
连接池	HikariCP	Spring Boot 默认	高性能、低延迟，Spring Boot 默认连接池，参数可精细配置
身份认证	JJWT	0.12.5	轻量的 JWT 实现，无状态认证，适合前后端共用凭证的场景
参数校验	Jakarta Validation	随父版本	以注解方式声明校验规则，配合全局异常统一处理
代码简化	Lombok	随父版本	以注解生成 getter/setter/构造器，减少样板代码
前端图表	Apache ECharts	5.5.0	成熟的数据可视化库，图表类型丰富、配置灵活
前端交互	原生 HTML/CSS/JS	—	不引入重型前端框架，降低复杂度，突出原生 Web 技术运用

表 3-1 技术选型与理由

3.3 分层设计

后端代码集中在 com.iot.qa 包下，按职责划分为若干子包。各层职责与关键类如下表所示。

包名	职责	关键类
controller	接收 HTTP 请求、参数校验、调用 Service、返回统一响应	AuthController、ChatController、PageController、PublicController
controller.admin	后台管理接口，路径统一以 /api/admin 开头	KnowledgeAdminController、CategoryAdminController、UserAdminController、SessionAdminController、StatsAdminController、ConfigAdminController

service	业务逻辑编排、事务控制	UserService、ChatService、KnowledgeService、CategoryService、ConfigService、StatsService
engine	问答引擎：策略选择、检索打分、失败降级	AnswerEngine、AnswerStrategy、LocalRetrievalStrategy、LlmStrategy、AnswerResult
mapper	数据访问接口，继承 BaseMapper，复杂查询用注解或 XML	QaKnowledgeMapper、QaMessageMapper、QaSessionMapper、QaCategoryMapper、SysUserMapper、SysConfigMapper
entity	数据库表映射实体，含逻辑删除与自动填充注解	SysUser、QaCategory、QaKnowledge、QaSession、QaMessage、SysConfig
dto	入参对象，集中声明校验规则	LoginRequest、RegisterRequest、ChatRequest、KnowledgeForm、CategoryForm、PasswordForm
vo	出参对象，屏蔽数据库细节	LoginVO、ChatResponse、DashboardVO
common	公共组件：响应封装、异常、鉴权、上下文	Result、PageResult、ResultCode、BusinessException、GlobalExceptionHandler、JwtUtil、AuthInterceptor、LoginUser、UserContext
config	框架配置	MybatisPlusConfig（分页插件）、MyMetaObjectHandler（字段填充）、WebMvcConfig（拦截器注册）
util	通用工具	PasswordUtil（密码加盐哈希）、TextUtils（文本相似度与转义）

表 3-2 分层职责与关键类

3.4 工程目录结构

```
20232653+物联网 2301/
├─ 源代码/
│  ├─ pom.xml                          # Maven 构建与依赖（Spring Boot 3.2.5 / Java 17）
│  ├─ sql/ai_qa.sql                    # 数据库建库、建表、索引与初始化数据脚本
│  └─ src/main/
│     ├─ java/com/iot/qa/
│     │  ├─ AiQaApplication.java       # 启动类（@SpringBootApplication + @MapperScan）
│     │  ├─ common/                    # Result / PageResult / JwtUtil / AuthInterceptor ...
│     │  ├─ config/                    # MybatisPlusConfig / MyMetaObjectHandler / WebMvcConfig
│     │  ├─ controller/                # 前台接口 + PageController
│     │  │  └─ admin/                  # 后台管理接口（6 个 Controller）
│     │  ├─ dto/                       # 入参对象（record + 校验注解）
│     │  ├─ engine/                    # 问答引擎与两种回答策略
│     │  ├─ entity/                    # 6 个表实体
│     │  ├─ mapper/                    # 6 个 Mapper 接口
│     │  ├─ service/                   # 6 个业务服务
│     │  ├─ util/                      # PasswordUtil / TextUtils
│     │  └─ vo/                        # 出参对象
│     └─ resources/
```

```
│        ├─ application.yml            # 数据源 / 连接池 / MyBatis-Plus / 业务自
定义配置
│        ├─ mapper/QaKnowledgeMapper.xml  # 知识条目分页联表查询 SQL
│        ├─ templates/                 # Thymeleaf 页面（前台 5 个 + 后台 7 个）
│        └─ static/
│           ├─ css/                    # style.css（前台）/ admin.css（后台）
│           └─ js/                     # common.js + 页面脚本 + admin/ 后台脚本
└─ 网站设计报告.docx                   # 本报告
```

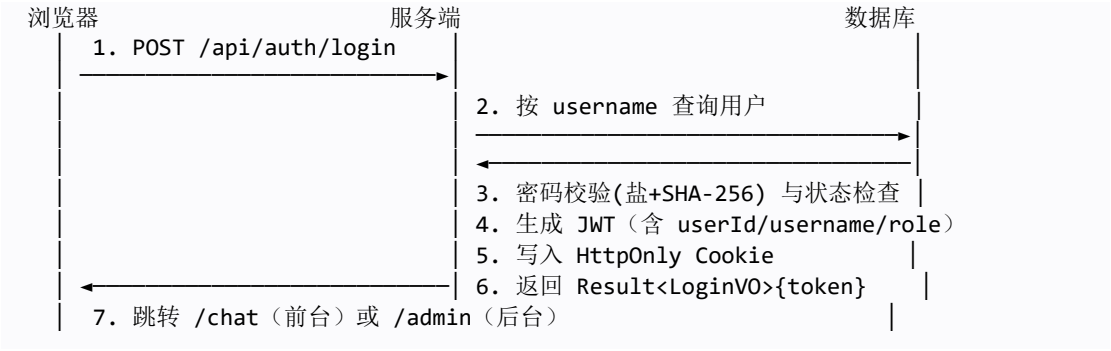
图 3-2 工程目录结构

3.5 核心业务流程

3.5.1 登录流程

登录是系统的入口环节。用户提交用户名与密码后，服务端先按用户名查询用户记录（逻辑删除的记录不会被查出），再用 PasswordUtil.matches 校验密码，随后检查账号状态，全部通过后签发 JWT 并同时写入响应体与 Cookie。

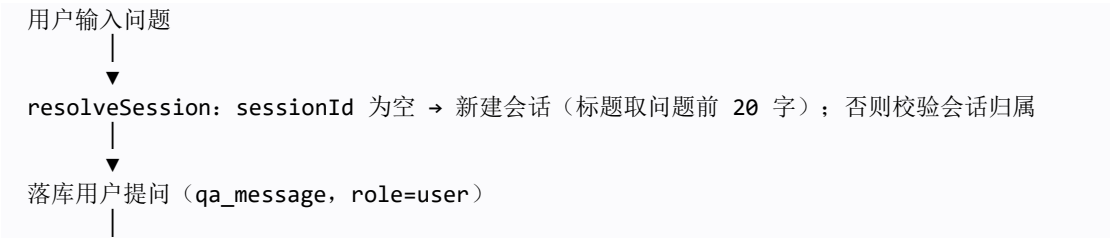
图 3-3 登录流程图

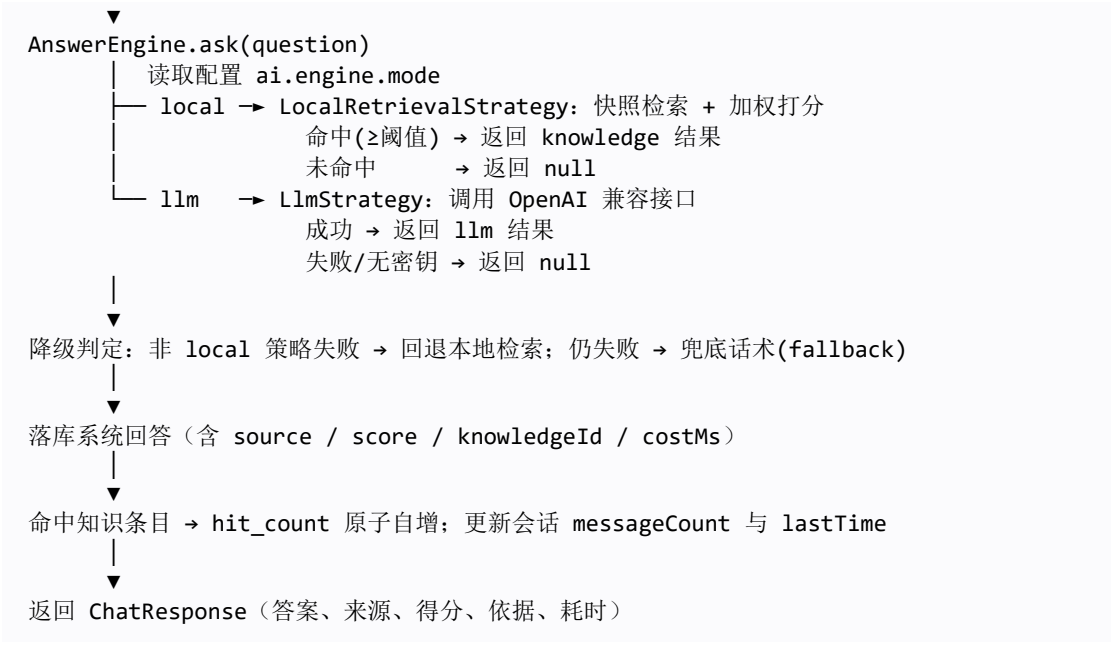


3.5.2 问答流程

问答流程是系统的核心业务链路，由 ChatService.chat 方法串联。它依次完成会话定位、用户提问落库、调用问答引擎、系统回答落库、命中计数累加与会话统计更新，整个过程处于同一个数据库事务中，保证「提问—回答」成对写入，不会出现只有提问没有回答的中间状态。

图 3-4 问答流程图（示意）

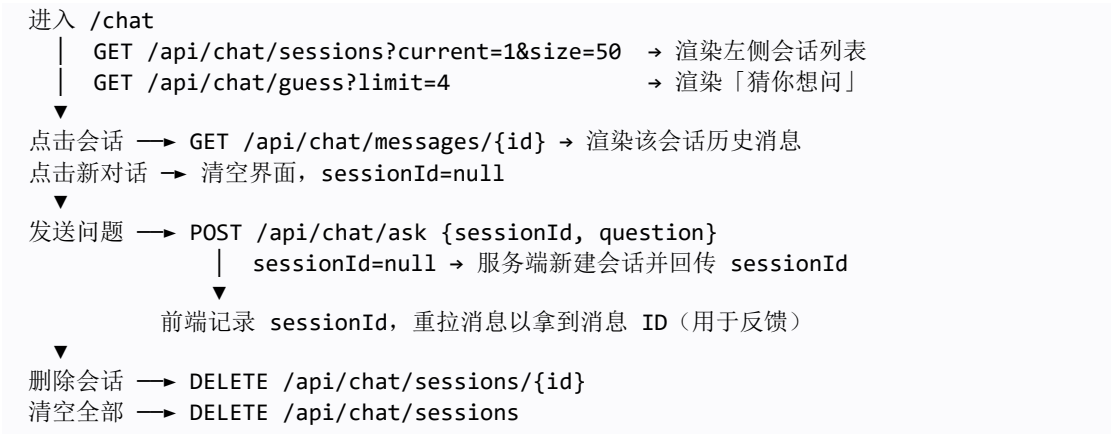




3.5.3 会话流程

会话用于把同一主题下的多轮问答聚合在一起。前端在左侧列出我的会话，点击某条会话时先读取该会话的历史消息，再把 sessionId 记入内存状态；之后每次提问都带上该 sessionId，服务端便能把新消息追加到同一会话下。若用户在未选择会话时直接提问，则 sessionId 为空，服务端自动创建新会话并把首条提问的前 20 个字符作为标题。

图 3-5 会话与历史流程



第四章 数据库设计

4.1 概念结构设计

数据库概念结构用 E-R 模型描述。系统涉及六个实体：用户（sys_user）、知识分类（qa_category）、知识条目（qa_knowledge）、问答会话（qa_session）、问答消息（qa_message）与系统参数（sys_config）。实体间的主要联系如下：

（1）一个用户可以拥有多个问答会话（1:N），会话通过 user_id 关联用户；

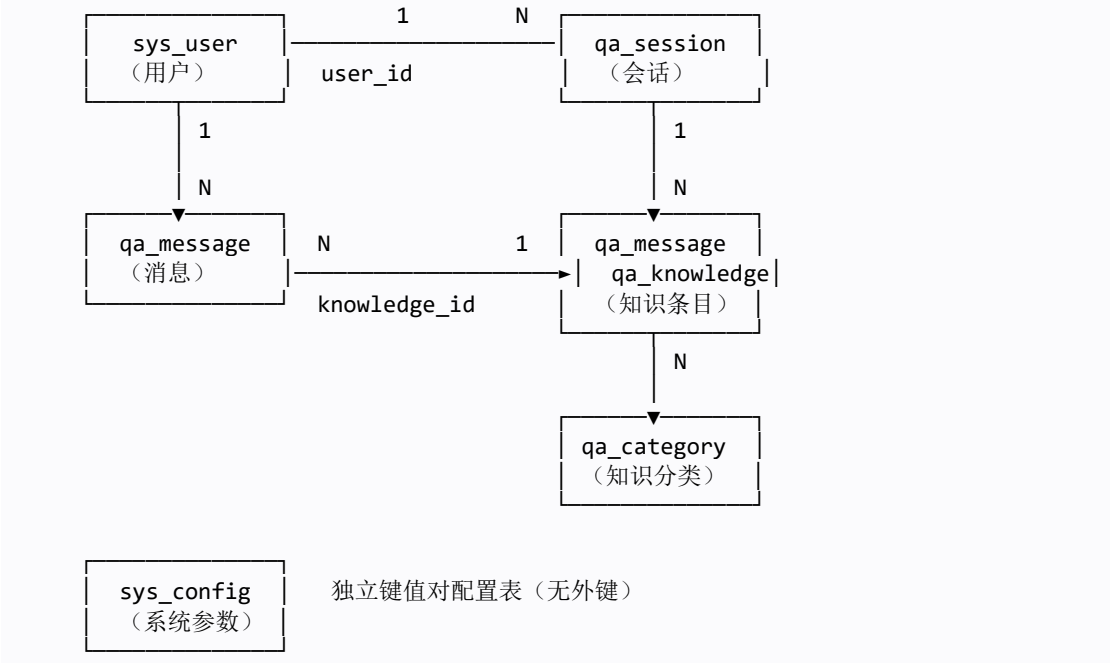
（2）一个会话包含多条问答消息（1:N），消息通过 session_id 关联会话；

（3）一条消息最多命中一个知识条目（N:1，可为空），通过 knowledge_id 关联；

（4）一个分类下可以有多个知识条目（1:N），知识条目通过 category_id 关联分类；

（5）系统参数（sys_config）为独立的键值对配置表，与其他实体无外键关联。

图 4-1 系统 E-R 关系图（示意）



说明：为简化实现并保证删除操作的可控性，物理表之间未建立数据库级外键约束，而是通过应用层的逻辑删除（deleted 字段）与业务校验维护引用一致性。例如删除分类前会先检查该分类下是否仍有未删除的知识条目。

4.2 逻辑结构设计

把 E-R 模型转换为关系模式，得到如下六个关系（下划线表示主键，箭头表示参照）：

sys_user（id，username，password，nickname，email，avatar，role，status，create_time，update_time，deleted）

qa_category（id，name，sort，description，icon，create_time，update_time，deleted）

qa_knowledge（id，category_id → qa_category.id，question，answer，keywords，hit_count，status，create_time，update_time，deleted）

qa_session（id，user_id → sys_user.id，title，message_count，last_time，create_time，update_time，deleted）

qa_message（id，session_id → qa_session.id，user_id → sys_user.id，role，content，knowledge_id → qa_knowledge.id，score，source，feedback，cost_ms，create_time，deleted）

sys_config（id，config_key，config_value，remark，create_time，update_time）

数据库统一采用 utf8mb4 字符集与 utf8mb4_general_ci 排序规则，存储引擎为 InnoDB，以支持事务与行级锁。除 sys_config 外，其余五张表均带有 deleted 逻辑删除标记。

序号	表名	中文名	说明
1	sys_user	系统用户表	存储普通用户与管理员账号
2	qa_category	知识分类表	知识条目的分类维度
3	qa_knowledge	知识库条目表	问答引擎检索的数据来源
4	qa_session	问答会话表	一次连续对话对应一条记录
5	qa_message	问答消息表	保存每一轮的提问与回答
6	sys_config	系统参数表	运行时可变配置的键值对存储

表 4-1 数据表清单

4.3 数据表详细设计

4.3.1 sys_user（系统用户表）

字段名	类型	约束	说明
id	BIGINT	PK，AUTO_INCREMENT	主键
username	VARCHAR(50)	NOT NULL，UNIQUE(uk_username)	登录名，唯一
password	VARCHAR(100)	NOT NULL	密码，格式为「盐值:哈希值」
nickname	VARCHAR(50)	NOT NULL，DEFAULT ''	昵称
email	VARCHAR(100)	DEFAULT NULL	邮箱
avatar	VARCHAR(255)	DEFAULT NULL	头像地址
role	VARCHAR(20)	NOT NULL，DEFAULT 'USER'	角色：ADMIN 管理员 / USER 普通用户
status	TINYINT	NOT NULL，DEFAULT 1	状态：1 正常 / 0 禁用
create_time	DATETIME	NOT NULL，DEFAULT CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	NOT NULL，ON UPDATE CURRENT_TIMESTAMP	更新时间
deleted	TINYINT	NOT NULL，DEFAULT 0	逻辑删除：0 未删除 / 1 已删除

表 4-2 sys_user 字段说明

4.3.2 qa_category（知识分类表）

字段名	类型	约束	说明
id	BIGINT	PK，AUTO_INCREMENT	主键
name	VARCHAR(50)	NOT NULL	分类名称
sort	INT	NOT NULL，DEFAULT 0	排序值，越小越靠前
description	VARCHAR(255)	DEFAULT NULL	分类描述
icon	VARCHAR(50)	DEFAULT 'icon-default'	图标标识（前端 CSS 类名）
create_time	DATETIME	NOT NULL，DEFAULT CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	NOT NULL，ON UPDATE CURRENT_TIMESTAMP	更新时间
deleted	TINYINT	NOT NULL，DEFAULT 0	逻辑删除：0 未删除 / 1 已删除

表 4-3 qa_category 字段说明

4.3.3 qa_knowledge（知识库条目表）

字段名	类型	约束	说明
id	BIGINT	PK，AUTO_INCREMENT	主键
category_id	BIGINT	DEFAULT NULL	所属分类（关联 qa_category.id）

question	VARCHAR(255)	NOT NULL	标准问题
answer	TEXT	NOT NULL	标准答案
keywords	VARCHAR(255)	DEFAULT ''	关键词，英文逗号分隔
hit_count	INT	NOT NULL，DEFAULT 0	被命中次数，用于热门统计
status	TINYINT	NOT NULL，DEFAULT 1	状态：1 启用 / 0 停用
create_time	DATETIME	NOT NULL，DEFAULT CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	NOT NULL，ON UPDATE CURRENT_TIMESTAMP	更新时间
deleted	TINYINT	NOT NULL，DEFAULT 0	逻辑删除：0 未删除 / 1 已删除

表 4-4 qa_knowledge 字段说明

4.3.4 qa_session（问答会话表）

字段名	类型	约束	说明
id	BIGINT	PK，AUTO_INCREMENT	主键
user_id	BIGINT	NOT NULL	所属用户（关联 sys_user.id）
title	VARCHAR(100)	NOT NULL，DEFAULT ''	会话标题，默认取首条提问前 20 字
message_count	INT	NOT NULL，DEFAULT 0	消息条数
last_time	DATETIME	DEFAULT NULL	最后活跃时间
create_time	DATETIME	NOT NULL，DEFAULT CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	NOT NULL，ON UPDATE CURRENT_TIMESTAMP	更新时间
deleted	TINYINT	NOT NULL，DEFAULT 0	逻辑删除：0 未删除 / 1 已删除

表 4-5 qa_session 字段说明

4.3.5 qa_message（问答消息表）

字段名	类型	约束	说明
id	BIGINT	PK，AUTO_INCREMENT	主键
session_id	BIGINT	NOT NULL	所属会话（关联 qa_session.id）
user_id	BIGINT	NOT NULL	所属用户
role	VARCHAR(20)	NOT NULL	角色：user 提问 / assistant 回答
content	TEXT	NOT NULL	消息正文
knowledge_id	BIGINT	DEFAULT NULL	命中的知识条目 ID
score	DOUBLE	DEFAULT NULL	相似度得分 0~1
source	VARCHAR(20)	DEFAULT NULL	答案来源：knowledge / llm / fallback
feedback	TINYINT	NOT NULL，DEFAULT 0	反馈：1 有帮助 / -1 没帮助 / 0 未评价

cost_ms	INT	DEFAULT NULL	引擎耗时（毫秒）
create_time	DATETIME	NOT NULL，DEFAULT CURRENT_TIMESTAMP	创建时间
deleted	TINYINT	NOT NULL，DEFAULT 0	逻辑删除：0 未删除 / 1 已删除

表 4-6 qa_message 字段说明

4.3.6 sys_config（系统参数表）

字段名	类型	约束	说明
id	BIGINT	PK，AUTO_INCREMENT	主键
config_key	VARCHAR(100)	NOT NULL，UNIQUE(uk_config_key)	参数键
config_value	VARCHAR(500)	DEFAULT ''	参数值
remark	VARCHAR(255)	DEFAULT NULL	参数说明
create_time	DATETIME	NOT NULL，DEFAULT CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	NOT NULL，ON UPDATE CURRENT_TIMESTAMP	更新时间

表 4-7 sys_config 字段说明

4.4 索引设计与优化说明

索引是保证查询性能的关键。系统在建表时即针对高频查询条件建立了索引，索引设计如下表所示。

表	索引名	类型	涉及字段	设计目的
sys_user	PRIMARY	主键	id	主键定位
sys_user	uk_username	唯一	username	登录与注册时按用户名精确查询，且保证唯一
sys_user	idx_status	普通	status	后台按状态筛选用户
sys_user	idx_role	普通	role	后台按角色筛选用户
qa_category	idx_sort	普通	sort	分类列表按排序值升序展示
qa_category	idx_deleted	普通	deleted	逻辑删除过滤
qa_knowledge	idx_category	普通	category_id	按分类统计条目数、按分类筛选
qa_knowledge	idx_status	普通	status	检索引擎只取启用条目
qa_knowledge	idx_hit_count	普通	hit_count	热门知识排序
qa_knowledge	idx_deleted	普通	deleted	逻辑删除过滤
qa_session	idx_user_last	联合	(user_id, last_time)	按用户查询会话并按最后活跃时间倒序
qa_session	idx_deleted	普通	deleted	逻辑删除过滤
qa_message	idx_session	普通	session_id	按会话查询消息列表
qa_message	idx_user	普通	user_id	按用户统计提问
qa_message	idx_role_time	联合	(role, create_time)	按角色+时间做趋势、时段统计

qa_message	idx_knowledge	普通	knowledge_id	回填命中的标准问题
sys_config	uk_config_key	唯一	config_key	按参数键精确读取，保证键唯一

表 4-8 索引设计一览

除索引外，系统还从以下几个层面进行了优化：

（1）分页与单页上限：通过 MyBatis-Plus 的 PaginationInnerInterceptor 统一实现物理分页，并设置 maxLimit=100，防止前端传入超大 size 导致数据库压力骤增。

（2）逻辑删除：使用 @TableLogic 注解，删除操作实际是把 deleted 置为 1，查询时自动追加 deleted=0 条件，既保留了数据可追溯性，又避免了物理删除带来的关联数据破坏。

（3）命中计数原子自增：热度累加使用 SQL 语句 hit_count = hit_count + 1 直接在数据库完成，而不是「查询—加一—写回」，避免了并发场景下的计数丢失。

（4）统计下推数据库：看板所需的提问趋势、来源分布、反馈分布、时段分布等统计，均以 GROUP BY / COUNT / DATE_FORMAT / HOUR 等聚合函数在数据库完成，只把聚合结果返回应用层。

（5）缓存：知识库检索使用 60 秒内存快照，系统参数使用 30 秒本地缓存，把高频读操作对数据库的压力降到最低。

4.5 初始化数据说明

sql/ai_qa.sql 脚本可重复执行（先 DROP DATABASE 再 CREATE），除建表与索引外还插入了完整的演示数据，便于系统开箱即用。初始化数据规模如下表所示。

数据对象	数量	说明
系统用户	3	admin（管理员，密码 admin123）、user（密码 123456）、demo（密码 123456）
知识分类	6	账号与登录、课程与学习、校园生活、技术支持、人工智能、其他问题
知识条目	27	分布在 6 个分类下，每条包含标准问题、标准答案、关键词与初始命中次数
问答会话	3	演示会话，用于后台看板与问答记录展示
问答消息	11	覆盖知识库命中与不同反馈值的演示消息

系统参数	7	问答模式、阈值、大模型地址/密钥/模型名、站点名称、首页公告

表 4-9 初始化数据规模

需要说明的是，脚本中初始用户的密码是使用 PasswordUtil 相同的算法（随机盐 + SHA-256）预先生成并写入的，因此可以直接用文档中给出的明文密码登录。系统参数表中的 ai.llm.apiKey 默认为空，这保证了系统在未配置大模型密钥时自动使用本地知识库检索，实现「开箱即用、离线可用」。

第五章 系统详细设计与实现

本章按照功能模块逐一说明实现思路，并给出关键代码片段。代码片段均取自项目真实源码，为便于排版对注释与空行做了少量精简，核心逻辑保持不变。

5.1 统一响应与全局异常处理

5.1.1 统一响应体 Result<T>

为保证前后端接口约定一致，所有接口统一返回 Result<T> 对象，包含业务状态码 code、提示信息 message、业务数据 data 以及服务端时间戳 timestamp。前端只需判断 code 是否为 200 即可确定请求是否成功，这大大简化了前端的处理逻辑。Result 提供了 success 与 error 两类静态工厂方法。

```
@Data
public class Result<T> implements Serializable {
    private Integer code;       // 业务状态码，200 表示成功
    private String  message;    // 提示信息
    private T       data;       // 业务数据
    private Long    timestamp = System.currentTimeMillis();

    public static <T> Result<T> success() { ... }
    public static <T> Result<T> success(T data) { ... }
    public static <T> Result<T> success(String message, T data) { ... }
    public static <T> Result<T> error(String message) { ... }
    public static <T> Result<T> error(ResultCode resultCode) { ... }
    public static <T> Result<T> error(Integer code, String message) { ... }
}
```

业务状态码集中在枚举 ResultCode 中管理，除常规的 200/400/401/403/404/500 外，还定义了用户与知识库相关的业务码，例如 1001 用户名已被注册、1002 用户不存在、1003 用户名或密码错误、1004 账号已被禁用、1005 原密码不正确、2001 知识条目不存在、2002 分类不存在、2003 分类下仍有知识条目无法删除、3001 会话不存在、4001 问答引擎调用失败。状态码与提示信息绑定，避免散落在各处的字符串。

5.1.2 分页结果 PageResult<T>

后端分页基于 MyBatis-Plus 的 IPage，但直接把它返回给前端会把持久层对象暴露出去。为此设计了 PageResult<T>，只保留 records、total、current、

size、pages 五个字段，通过 of(IPage) 静态方法完成转换，从而隔离持久层细节。

5.1.3 全局异常处理

GlobalExceptionHandler 使用 @RestControllerAdvice 集中拦截各类异常，把它们统一转换为 Result 响应，避免异常堆栈直接暴露给前端。它处理的异常包括：自定义业务异常 BusinessException、@RequestBody 参数校验失败 MethodArgumentNotValidException、表单绑定失败 BindException、约束校验失败 ConstraintViolationException、缺少请求参数、请求体格式错误、请求方法不支持、找不到处理器，以及兜底的 Exception。

```
@Slf4j
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BusinessException.class)
    public Result<Void> handleBusinessException(BusinessException e) {
        log.warn("业务异常: {}", e.getMessage());
        return Result.error(e.getCode(), e.getMessage());
    }

    /** @Valid 校验 @RequestBody 失败：把各字段的错误信息用「；」拼接后返回 */
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public Result<Void>
handleMethodArgumentNotValid(MethodArgumentNotValidException e) {
        String message = e.getBindingResult().getFieldErrors().stream()
                .map(FieldError::getDefaultMessage)
                .collect(Collectors.joining("；"));
        return Result.error(ResultCode.PARAM_ERROR.getCode(), message);
    }

    @ExceptionHandler(Exception.class)
    public Result<Void> handleException(Exception e) {
        log.error("系统异常", e);
        return Result.error("服务器开小差了，请稍后重试");
    }
}
```

这样的设计带来两个好处：一是错误提示统一由服务端给出，前端只需展示 message 即可；二是未知异常不会泄露堆栈信息，只记录到服务端日志，符合安全性要求。

5.2 用户认证与权限控制

5.2.1 密码加盐哈希

PasswordUtil 为每个用户生成 16 位十六进制随机盐，再对「盐值 + 明文密码」做 SHA-256 摘要，最终以「盐值:哈希值」的格式存储。校验时从存储串中截取出盐值，用同样的算法重新计算摘要并与存储的哈希比较。由于每个用户的盐值不同，即使两个用户密码相同，存储的密文也不同，从而有效抵御彩虹表攻击。

```
public final class PasswordUtil {
    private static final SecureRandom RANDOM = new SecureRandom();

    /** 生成 16 位十六进制随机盐 */
    public static String randomSalt() {
        byte[] bytes = new byte[8];
        RANDOM.nextBytes(bytes);
        return toHex(bytes);
    }

    /** 加密，返回 salt:hash */
    public static String encode(String rawPassword) {
        String salt = randomSalt();
        return salt + ":" + sha256(salt + rawPassword);
    }

    /** 校验原始密码是否匹配已存储的密文 */
    public static boolean matches(String rawPassword, String encodedPassword)
{
        if (rawPassword == null || encodedPassword == null) return false;
        int idx = encodedPassword.indexOf(':');
        if (idx <= 0) return false;
        String salt = encodedPassword.substring(0, idx);
        String hash = encodedPassword.substring(idx + 1);
        return sha256(salt + rawPassword).equals(hash);
    }
}
```

5.2.2 JWT 生成与解析

JwtUtil 负责签发与解析 JWT。签发时把用户 ID 放入 subject，把用户名与角色放入自定义 claim，并设置签发时间与过期时间（由配置 ai.jwt.expire 指定，默认 7200 秒）；解析时校验签名与有效期，失败则返回 null，由拦截器统一处理。密钥来自配置 ai.jwt.secret，由于 HS256 要求密钥长度不少于 32 字节，配置中使用了足够长的字符串。

```
@Component
public class JwtUtil {
    @Value("${ai.jwt.secret}") private String secret;
```

```
    @Value("${ai.jwt.expire}") private Long expire;

    private SecretKey getKey() {
        return Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }

    public String createToken(Long userId, String username, String role) {
        Date now = new Date();
        Date expiry = new Date(now.getTime() + expire * 1000L);
        return Jwts.builder()
                .subject(String.valueOf(userId))
                .claim("username", username)
                .claim("role", role)
                .issuedAt(now).expiration(expiry)
                .signWith(getKey()).compact();
    }

    public LoginUser parseToken(String token) {
        if (token == null || token.isBlank()) return null;
        try {
            Claims claims = Jwts.parser().verifyWith(getKey()).build()
                    .parseSignedClaims(token).getPayload();
            LoginUser user = new LoginUser();
            user.setId(Long.valueOf(claims.getSubject()));
            user.setUsername(claims.get("username", String.class));
            user.setRole(claims.get("role", String.class));
            return user;
        } catch (JwtException | IllegalArgumentException e) {
            return null;
        }
    }
}
```

5.2.3 登录拦截器与权限校验

AuthInterceptor 是一个 HandlerInterceptor。它优先从请求头（默认 Authorization，前缀 "Bearer "）读取 token，其次从名为 token 的 Cookie 中读取，这样页面跳转（服务端渲染）与 AJAX 接口调用可以共用同一套凭证。解析成功后，如果访问的是 /admin 或 /api/admin 开头的路径，还会额外校验角色必须为 ADMIN。校验通过后把登录用户写入 UserContext（ThreadLocal），请求结束时由 afterCompletion 清理，防止线程复用导致的数据串号。

```
@Component
@RequiredArgsConstructor
public class AuthInterceptor implements HandlerInterceptor {

    private final JwtUtil jwtUtil;

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse
response, Object handler) throws Exception {
        String token = resolveToken(request);                // 请求头优先，其
次 Cookie
        LoginUser loginUser = jwtUtil.parseToken(token);
        if (loginUser == null) { reject(request, response,
```

```
ResultCode.UNAUTHORIZED); return false; }

        String path = request.getRequestURI();
        if ((path.startsWith("/admin") || path.startsWith("/api/admin"))
&& !loginUser.isAdmin()) {
            reject(request, response, ResultCode.FORBIDDEN);
            return false;
        }
        UserContext.set(loginUser);
        return true;
    }

    @Override
    public void afterCompletion(HttpServletRequest request,
HttpServletResponse response,
                                Object handler, Exception ex) {
        UserContext.clear();
    }
}
```

拒绝访问时区分了两类响应：若请求路径以 /api/ 开头，则直接写出 JSON 格式的 401/403 响应；否则（页面请求）重定向到 /login 或 /admin/login。拦截器的注册在 WebMvcConfig 中完成，拦截 /api/**、/admin、/admin/**、/chat、/history，并放行登录、注册、注销、公开接口、后台登录页与静态资源等路径。

5.3 问答引擎（核心算法）

问答引擎是本系统最具技术含量的部分，采用策略模式设计，由 AnswerEngine 统一入口，向下挂载两个可互换的策略实现：本地知识库检索策略 LocalRetrievalStrategy 与大模型策略 LlmStrategy。引擎的职责是「选择策略 + 失败降级 + 兜底话术」，具体如何回答则交给策略实现，符合开闭原则。

5.3.1 策略接口与引擎调度

AnswerStrategy 接口只声明两个方法：mode() 返回策略标识（与配置项 ai.engine.mode 对应），answer(question) 返回回答结果或 null。AnswerEngine 在构造时通过 Spring 注入所有 AnswerStrategy 实现，并以 mode 为键放入 Map。调用 ask 时先读取配置的 mode 取出主策略，执行成功后直接返回；若主策略失败（返回 null 或抛异常），且主策略不是 local，则降级调用 local 策略；若仍然失败，则返回兜底话术。

```
@Slf4j
@Service
public class AnswerEngine {
```

```
    private static final String FALLBACK_TEMPLATE =
            "抱歉，我暂时没有找到与「%s」相关的标准答案。\n 你可以：\n"
          + "1. 换一种更具体的说法，例如把「怎么弄」改成「如何注册账号」；\n"
          + "2. 在下方「猜你想问」里选择一个已有问题；\n"
          + "3. 联系管理员把该问题补充进知识库。";

    private final Map<String, AnswerStrategy> strategies = new HashMap<>();
    private final ConfigService configService;

    public AnswerEngine(List<AnswerStrategy> strategyList, ConfigService
configService) {
        this.configService = configService;
        for (AnswerStrategy strategy : strategyList) {
            strategies.put(strategy.mode(), strategy);
        }
        log.info("问答引擎已加载策略: {}", strategies.keySet());
    }

    public AnswerResult ask(String question) {
        String mode = configService.get("ai.engine.mode", "local");
        AnswerStrategy primary = strategies.get(mode);
        if (primary != null) {
            try {
                AnswerResult result = primary.answer(question);
                if (result != null) return result;
            } catch (Exception e) {
                log.warn("策略[{}]执行异常: {}", mode, e.getMessage());
            }
        }
        // 降级：非本地策略失败时回退到知识库检索
        if (!"local".equals(mode)) {
            AnswerStrategy local = strategies.get("local");
            if (local != null) {
                try {
                    AnswerResult result = local.answer(question);
                    if (result != null) { result.setSource("knowledge");
return result; }
                } catch (Exception e) {
                    log.warn("降级到本地检索同样失败: {}", e.getMessage());
                }
            }
        }
        return AnswerResult.ofFallback(String.format(FALLBACK_TEMPLATE,
question));
    }
}
```

AnswerResult 是引擎的输出载体，包含回答内容 answer、来源 source（knowledge/llm/fallback）、相似度得分 score、命中的知识条目 ID knowledgeId 以及命中的标准问题 matchedQuestion，并通过 ofKnowledge、ofLlm、ofFallback 三个静态工厂方法构造。

5.3.2 文本表示：字符二元组 + 杰卡德相似度

中文没有天然的空格分词，若直接按字匹配会丢失语序信息，若引入分词组件又会增加部署成本与不确定性。为此 TextUtils 采用字符二元组（bigram）表示文本：先把文本归一化（去除标点、空白与控制字符，英文转小写），再以滑窗方式切出所有长度为 2 的连续子串构成集合。文本相似度则用杰卡德相似度衡量，即两个二元组集合的交集大小与并集大小之比。

```
public final class TextUtils {
    /** 标点、空白、控制字符 */
    private static final Pattern NOISE =
            Pattern.compile("[\\p{Punct}\\s\\u3000-\\u303F\\uFF00-\\uFFEF]+");

    /** 归一化：去标点空白、英文转小写 */
    public static String normalize(String text) {
        return text == null ? "" :
NOISE.matcher(text.toLowerCase()).replaceAll("");
    }

    /** 生成字符二元组集合 */
    public static Set<String> bigrams(String text) {
        String normalized = normalize(text);
        Set<String> grams = new HashSet<>();
        if (normalized.isEmpty()) return grams;
        if (normalized.length() == 1) { grams.add(normalized); return grams; }
        for (int i = 0; i + 2 <= normalized.length(); i++) {
            grams.add(normalized.substring(i, i + 2));
        }
        return grams;
    }

    /** 杰卡德相似度：交集大小 / 并集大小 */
    public static double jaccard(Set<String> a, Set<String> b) {
        if (a.isEmpty() || b.isEmpty()) return 0D;
        Set<String> intersection = new HashSet<>(a);
        intersection.retainAll(b);
        Set<String> union = new HashSet<>(a);
        union.addAll(b);
        return (double) intersection.size() / union.size();
    }

    public static double similarity(String textA, String textB) {
        return jaccard(bigrams(textA), bigrams(textB));
    }
}
```

举例来说，问题「图书馆开放时间是什么时候」与标准问题「图书馆开放时间是什么时候？」在归一化后完全一致，二元组集合相同，杰卡德相似度为 1；而用户提问「图书馆几点开门」虽然与标准问题字面不同，但由于共享「图书」「书馆」等二元组，仍会得到一定相似度，再叠加关键词命中（关键词中包含「图书馆」「几点开门」）后即可越过阈值命中。

5.3.3 关键词命中率

除整体相似度外，关键词命中率是另一个重要信号。知识条目的 keywords 字段用逗号分隔若干关键词，keywordHitRate 统计用户问题中命中了多少个关键词，除以期望命中个数（常量 EXPECTED_KEYWORD_HITS = 3），并截断到 1.0。也就是说，命中 3 个及以上关键词即视为满分。

```
/** 关键词命中率：命中数 / 期望命中数 */
public static double keywordHitRate(String question, String keywords, int
expectedHits) {
    if (keywords == null || keywords.isBlank()) return 0D;
    String normalizedQuestion = normalize(question);
    long hits = Arrays.stream(keywords.split("[,，、|\\s]+"))
            .map(TextUtils::normalize)
            .filter(k -> !k.isEmpty())
            .distinct()
            .filter(normalizedQuestion::contains)
            .count();
    if (hits == 0) return 0D;
    return Math.min(1D, (double) hits / expectedHits);
}
```

5.3.4 本地检索打分公式

LocalRetrievalStrategy 对知识库中每一条启用条目计算综合得分，公式为：

score = 0.45 × 关键词命中率 + 0.40 × 问题文本相似度 + 0.15 × 关键词文本相似度

其中权重 0.45 给关键词命中率，因为关键词是管理员针对该条目人工提炼的核心词，命中即强信号；0.40 给问题文本相似度，衡量用户问法与标准问题的整体接近程度；0.15 给关键词文本相似度，作为辅助信号，用于在关键词命中相近时进一步区分。三者加权求和后取值范围为 0~1，取分数最高且不低于阈值（默认 0.25）的条目作为答案，得分保留三位小数。

```
@Override
public AnswerResult answer(String question) {
    if (question == null || question.isBlank()) return null;
    double threshold = configService.getDouble("ai.engine.threshold", 0.25D);
    List<QaKnowledge> items = currentSnapshot();
    if (items.isEmpty()) return null;

    QaKnowledge best = null;
    double bestScore = 0D;
    for (QaKnowledge item : items) {
        double score = score(question, item);
        if (score > bestScore) { bestScore = score; best = item; }
    }
    if (best == null || bestScore < threshold) {
        log.debug("本地检索未命中，最高分={}, 阈值={}", bestScore, threshold);
```

```
        return null;
    }
    return AnswerResult.ofKnowledge(best.getAnswer(), round(bestScore),
                                    best.getId(), best.getQuestion());
}

/** 单条知识打分 */
private double score(String question, QaKnowledge item) {
    double keywordRate = TextUtils.keywordHitRate(question, item.getKeywords(),
EXPECTED_KEYWORD_HITS);
    double questionSim = TextUtils.similarity(question, item.getQuestion());
    double keywordSim  = TextUtils.similarity(question, item.getKeywords());
    return 0.45D * keywordRate + 0.40D * questionSim + 0.15D * keywordSim;
}
```

5.3.5 知识库内存快照缓存

问答属于高频读操作。如果每次提问都全表扫描知识库，数据库压力会随访问量线性增长。为此 LocalRetrievalStrategy 在内存中维护一份启用条目的快照，有效期 60 秒（常量 SNAPSHOT_TTL_MS）；只有超过有效期才重新查库，且用双重检查加锁避免并发重复加载。当管理员在后台新增、修改、删除或启停知识条目时，KnowledgeService 会调用 invalidate() 立即让快照失效，保证新答案能被马上检索到。这种「读多写少 + 定时失效 + 写时主动失效」的策略，把「每次提问一次全表扫描」降为「每分钟最多一次」，兼顾了性能与实时性。

```
private volatile List<QaKnowledge> snapshot = Collections.emptyList();
private volatile long snapshotLoadedAt = 0L;
private final Object snapshotLock = new Object();

private List<QaKnowledge> currentSnapshot() {
    if (System.currentTimeMillis() - snapshotLoadedAt < SNAPSHOT_TTL_MS) {
        return snapshot;
    }
    synchronized (snapshotLock) {
        if (System.currentTimeMillis() - snapshotLoadedAt < SNAPSHOT_TTL_MS) {
            return snapshot;                                    // 双重检查
        }
        try {
            snapshot = knowledgeMapper.selectAllEnabled();
            snapshotLoadedAt = System.currentTimeMillis();
        } catch (Exception e) {
            log.warn("加载知识库快照失败，沿用旧数据: {}", e.getMessage());
            snapshotLoadedAt = System.currentTimeMillis();
        }
        return snapshot;
    }
}

/** 知识条目发生变化时由业务层调用，使快照立即失效 */
public void invalidate() {
    snapshotLoadedAt = 0L;
}
```

5.3.6 大模型策略与失败降级

LlmStrategy 调用 OpenAI 兼容的 /chat/completions 接口。它从 sys_config 读取接口地址、密钥与模型名，未配置密钥时直接返回 null（即放弃该策略），由引擎降级到本地检索。请求体包含 system 提示词、用户问题、temperature 与 max_tokens；解析响应时取 choices[0].message.content。任何网络异常或解析失败都被捕获并返回 null，保证不会因为外部服务问题导致问答功能整体不可用。

```
@Override
public AnswerResult answer(String question) {
    String apiKey = configService.get("ai.llm.apiKey", "");
    if (apiKey.isBlank()) {
        log.debug("未配置大模型 API Key，跳过 LLM 策略");
        return null;                                    // 触发引擎降级
    }
    String baseUrl = configService.get("ai.llm.baseUrl",
"https://api.openai.com/v1");
    String model   = configService.get("ai.llm.model", "gpt-4o-mini");
    String url = baseUrl.replaceAll("/+$", "") + "/chat/completions";

    Map<String, Object> body = Map.of(
            "model", model,
            "messages", List.of(
                    Map.of("role", "system", "content", SYSTEM_PROMPT),
                    Map.of("role", "user", "content", question)),
            "temperature", 0.7,
            "max_tokens", 800);

    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_JSON);
    headers.setBearerAuth(apiKey);
    try {
        ResponseEntity<String> response = restTemplate.postForEntity(
                url, new HttpEntity<>(body, headers), String.class);
        String answer = extractContent(response.getBody());   //
choices[0].message.content
        return answer == null ? null : AnswerResult.ofLlm(answer);
    } catch (Exception e) {
        log.warn("调用大模型接口失败: {}", e.getMessage());
        return null;
    }
}
```

由此形成一条清晰的三级保障链：配置指定的策略（可能是 llm）→ 本地知识库检索 → 兜底话术。无论外部环境如何变化，用户始终能得到一个明确的回应，不会出现「转圈无响应」的情况。

5.4 会话与问答历史

ChatService.chat 方法在一次数据库事务中完成整轮问答的落库与统计更新。它先通过 resolveSession 定位会话：若请求带 sessionId，则查出会话并校验归属（不属于当前用户时抛 403）；若未带 sessionId，则新建会话，标题取问题的前 20 个字符。随后依次落库用户提问、调用引擎、落库系统回答、命中则累加热度、更新会话的消息数与最后活跃时间，最后组装 ChatResponse 返回。

```
@Transactional(rollbackFor = Exception.class)
public ChatResponse chat(Long userId, ChatRequest request) {
    long start = System.currentTimeMillis();
    QaSession session = resolveSession(userId, request);

    // 1. 落库用户提问
    QaMessage userMessage = new QaMessage();
    userMessage.setSessionId(session.getId());
    userMessage.setUserId(userId);
    userMessage.setRole("user");
    userMessage.setContent(request.question());
    userMessage.setDeleted(0);
    messageMapper.insert(userMessage);

    // 2. 交给问答引擎求解
    AnswerResult answerResult = answerEngine.ask(request.question());

    // 3. 落库系统回答
    int costMs = (int) (System.currentTimeMillis() - start);
    QaMessage aiMessage = new QaMessage();
    aiMessage.setSessionId(session.getId());
    aiMessage.setUserId(userId);
    aiMessage.setRole("assistant");
    aiMessage.setContent(answerResult.getAnswer());
    aiMessage.setKnowledgeId(answerResult.getKnowledgeId());
    aiMessage.setScore(answerResult.getScore());
    aiMessage.setSource(answerResult.getSource());
    aiMessage.setFeedback(0);
    aiMessage.setCostMs(costMs);
    aiMessage.setDeleted(0);
    messageMapper.insert(aiMessage);

    // 4. 命中知识条目则累计热度（SQL 原子自增）
    if (answerResult.getKnowledgeId() != null) {
        knowledgeService.incrementHit(answerResult.getKnowledgeId());
    }

    // 5. 更新会话统计信息
    session.setMessageCount((session.getMessageCount() == null ? 0 :
session.getMessageCount()) + 2);
    session.setLastTime(LocalDateTime.now());
    sessionMapper.updateById(session);

    ChatResponse response = new ChatResponse();
    response.setSessionId(session.getId());
    response.setSessionTitle(session.getTitle());
    response.setQuestion(request.question());
    response.setAnswer(answerResult.getAnswer());
```

```
    response.setSource(answerResult.getSource());
    response.setScore(answerResult.getScore());
    response.setKnowledgeId(answerResult.getKnowledgeId());
    response.setMatchedQuestion(answerResult.getMatchedQuestion());
    response.setCostMs(costMs);
    response.setCreateTime(LocalDateTime.now());
    return response;
}
```

listMessages 用于读取会话内的消息列表，它按 id 升序返回消息，并对带有 knowledgeId 的助手消息回填命中的标准问题（matchedQuestion），便于前端展示「依据」。删除会话与清空会话同样在事务中完成，会同时逻辑删除会话及其全部消息。答案反馈接口 feedback 会校验消息归属，防止用户对他人的回答进行评价。

5.5 知识库与分类管理

5.5.1 知识库管理

知识库管理由 KnowledgeAdminController 提供四个接口：分页查询（/page）、保存（/save）、删除（DELETE /{id}）与启停（PUT /{id}/status）。分页查询使用自定义 XML 中的 selectPageWithCategory，左连接分类表带出分类名称，并支持关键词（匹配问题、关键词、答案）、分类与状态三个过滤条件。

```
<select id="selectPageWithCategory"
resultType="com.iot.qa.entity.QaKnowledge">
    SELECT k.id, k.category_id, k.question, k.answer, k.keywords,
           k.hit_count, k.status, k.create_time, k.update_time,
           c.name AS category_name
    FROM qa_knowledge k
             LEFT JOIN qa_category c ON c.id = k.category_id AND c.deleted = 0
    WHERE k.deleted = 0
    <if test="categoryId != null"> AND k.category_id = #{categoryId} </if>
    <if test="status != null">     AND k.status = #{status} </if>
    <if test="keyword != null and keyword != ''">
        AND (k.question LIKE CONCAT('%', #{keyword}, '%')
          OR k.keywords LIKE CONCAT('%', #{keyword}, '%')
          OR k.answer   LIKE CONCAT('%', #{keyword}, '%'))
    </if>
    ORDER BY k.update_time DESC, k.id DESC
</select>
```

保存操作区分新增与修改：新增时命中次数置 0，修改时先校验条目存在。无论新增、修改、删除还是启停，都会调用

localRetrievalStrategy.invalidate() 使检索引擎的内存快照失效。命中计数使用 SQL 原子自增，避免并发下的计数丢失。

```
/** 命中计数 +1：使用 SQL 原子自增而非「查出来再写回去」，避免并发下的计数丢失 */
public void incrementHit(Long id) {
    update(new UpdateWrapper<QaKnowledge>()
            .setSql("hit_count = hit_count + 1")
            .eq("id", id));
}
```

5.5.2 分类管理

分类管理提供列表（附带各分类下的知识条目数量）、下拉选项、保存与删除四个接口。列表查询先用 LambdaQueryWrapper 按 sort 升序取出全部分类，再调用 countGroupByCategory 得到各分类的条目数，在内存中把两者合并。删除分类前会先统计该分类下的知识条目数量，若大于 0 则抛出业务异常，提示「该分类下仍有知识条目，无法删除」，从而在应用层维护引用一致性。

```
/** 删除分类，分类下仍有知识条目时禁止删除 */
@Transactional(rollbackFor = Exception.class)
public void removeCategory(Long id) {
    if (getById(id) == null) throw new
BusinessException(ResultCode.CATEGORY_NOT_FOUND);
    long knowledgeCount = knowledgeMapper.selectCount(
            new
LambdaQueryWrapper<QaKnowledge>().eq(QaKnowledge::getCategoryId, id));
    if (knowledgeCount > 0) throw new
BusinessException(ResultCode.CATEGORY_IN_USE);
    removeById(id);
}
```

5.6 用户管理与系统配置

5.6.1 用户管理

UserService 承担了注册、登录、资料维护与后台用户管理等多重职责。注册时先校验两次密码一致性与用户名唯一性，再把密码加盐哈希后入库，角色固定为 USER。登录时按用户名查询用户（限定 LIMIT 1），校验密码与账号状态，通过后签发 token。后台用户管理支持分页查询（关键词匹配用户名或昵称、角色过滤）、启用/禁用、重置密码与删除；控制器层还做了保护，不允许禁用或删除当前登录的管理员自身。

```
@PutMapping("/{id}/status")
public Result<Void> updateStatus(@PathVariable Long id, @RequestParam Integer
```

```
status) {
    if (id.equals(UserContext.getUserId())) {
        return Result.error("不能禁用当前登录的管理员账号");
    }
    userService.updateStatus(id, status);
    return Result.success("状态已更新", null);
}
```

5.6.2 系统配置

系统参数存放在 sys_config 表中，由 ConfigService 统一读写。为避免每次问答都查库，ConfigService 在内存中维护一份参数缓存，有效期 30 秒；读取时若缓存未过期直接返回，过期后在同步块中重新加载，加载失败时保留旧缓存并延迟一个周期重试。管理员保存参数后调用 refresh() 强制刷新缓存，因此修改配置无需重启服务即可生效。

```
@Slf4j
@Service
public class ConfigService {
    private static final long TTL_MS = 30_000L;
    private final SysConfigMapper sysConfigMapper;
    private final Map<String, String> cache = new ConcurrentHashMap<>();
    private volatile long loadedAt = 0L;

    public String get(String key, String defaultValue) {
        loadIfExpired();
        String value = cache.get(key);
        return (value == null || value.isBlank()) ? defaultValue : value;
    }

    public void save(String key, String value) {
        SysConfig exists = sysConfigMapper.selectOne(new
LambdaQueryWrapper<SysConfig>()
                .eq(SysConfig::getConfigKey, key).last("LIMIT 1"));
        if (exists == null) {
            SysConfig config = new SysConfig();
            config.setConfigKey(key);
            config.setConfigValue(value);
            sysConfigMapper.insert(config);
        } else {
            exists.setConfigValue(value);
            sysConfigMapper.updateById(exists);
        }
        refresh();                      // 强制刷新，立即生效
    }
}
```

5.7 统计看板实现

StatsService.dashboard() 聚合了看板所需的全部指标。总数类指标（用户数、知识条目数、会话数、提问数、今日提问数）通过 MyBatis-Plus 的 count

与条件查询完成；趋势与分布类指标则全部下推到数据库，用 GROUP BY 完成聚合，只把聚合结果返回应用层。

```
/** 最近 N 天的提问量趋势 */
@Select("SELECT DATE_FORMAT(create_time, '%Y-%m-%d') AS day, COUNT(*) AS num "
      + "FROM qa_message WHERE role = 'user' AND deleted = 0 "
      + "AND create_time >= DATE_SUB(CURDATE(), INTERVAL #{days} DAY) "
      + "GROUP BY day ORDER BY day")
List<Map<String, Object>> countDailyTrend(@Param("days") int days);

/** 按答案来源统计占比 */
@Select("SELECT source AS source, COUNT(*) AS num FROM qa_message "
      + "WHERE role = 'assistant' AND deleted = 0 GROUP BY source")
List<Map<String, Object>> countGroupBySource();

/** 按小时统计提问分布（0-23 时） */
@Select("SELECT HOUR(create_time) AS hour, COUNT(*) AS num FROM qa_message "
      + "WHERE role = 'user' AND deleted = 0 GROUP BY hour ORDER BY hour")
List<Map<String, Object>> countGroupByHour();
```

在服务层，命中率与满意度由聚合结果二次计算：命中率 = 来源为 knowledge 的回答数 / 全部回答数；满意度 = 有帮助评价数 /（有帮助 + 没帮助）评价数。结果保留三位小数。分类知识量分布则复用分类服务的 listWithCount，得到「分类名—条目数」列表。最终所有指标组装为 DashboardVO 返回，由前端 ECharts 渲染成图表。

```
/** 命中率 = 知识库回答数 / 全部回答数 */
private Double calculateHitRate(List<Map<String, Object>> sourceStats) {
    long total = 0L, hit = 0L;
    for (Map<String, Object> row : sourceStats) {
        long num = ((Number) row.get("num")).longValue();
        total += num;
        if ("knowledge".equals(String.valueOf(row.get("source")))) hit += num;
    }
    return total == 0 ? 0D : Math.round(hit * 1000D / total) / 1000D;
}

/** 满意度 = 有帮助 /（有帮助 + 没帮助） */
private Double calculateSatisfiedRate(List<Map<String, Object>> feedbackStats)
{
    long positive = 0L, negative = 0L;
    for (Map<String, Object> row : feedbackStats) {
        int feedback = ((Number) row.get("feedback")).intValue();
        long num = ((Number) row.get("num")).longValue();
        if (feedback == 1) positive = num;
        else if (feedback == -1) negative = num;
    }
    long rated = positive + negative;
    return rated == 0 ? 0D : Math.round(positive * 1000D / rated) / 1000D;
}
```

第六章 前端设计与实现

6.1 页面结构与样式体系

前端采用「Thymeleaf 渲染页面骨架 + 原生 JavaScript 异步取数」的模式。服务端只负责返回静态的 HTML 骨架，所有动态数据（站点信息、问答结果、列表数据、统计指标）都通过 fetch 调用 REST 接口获取并渲染。这样做既保证了首屏的结构稳定，又让页面具备单页应用般的交互体验，同时避免了引入 React/Vue 等框架带来的构建复杂度。

前台共 5 个页面：index.html（首页）、login.html（登录）、register.html（注册）、chat.html（智能问答）、history.html（问答历史）；后台共 7 个页面：admin/login.html（管理员登录）、dashboard.html（数据看板）、knowledge.html（知识库管理）、category.html（分类管理）、user.html（用户管理）、message.html（问答记录）、config.html（系统配置）。

页面	路径	主要内容
首页	/	导航栏、Hero 区（站点名称/公告/平台规模）、功能卡片、热门问题、页脚
登录	/login	登录表单、错误提示区、演示账号提示
注册	/register	注册表单（用户名/昵称/邮箱/密码/确认密码）
智能问答	/chat	左侧会话侧栏、消息气泡区、猜你想问、输入框
问答历史	/history	会话卡片列表、展开对话、删除与清空
管理员登录	/admin/login	后台登录表单
数据看板	/admin	7 个统计卡、4 张图表、2 张 TOP10 榜单
知识库管理	/admin/knowledge	筛选工具栏、数据表格、分页、新增/编辑弹窗
分类管理	/admin/category	分类表格、新增/编辑弹窗
用户管理	/admin/user	用户表格、分页、重置密码弹窗
问答记录	/admin/message	会话表格、分页、对话详情弹窗
系统配置	/admin/config	引擎参数表单、站点信息表单、参数明细表

表 6-1 页面清单

样式体系方面，前台使用 style.css，后台使用 admin.css。前台样式表在 :root 中定义了统一的设计变量，包括品牌主色 --brand（#5b5bf5）、辅助色 --brand-2（#8b5cf6）、文字色 --ink（#1b1f36）与 --ink-2、弱化色 --muted（#8a90a8）、背景色 --bg（#f4f6fc）、卡片色、边框色、圆角（--radius 16px / --radius-sm 10px）与三级阴影（--shadow-sm / --shadow / --shadow-lg），以及中文字体栈。所有组件都引用这些变量，保证了整体视觉的一致性与可维护性。

```
:root {
    --brand: #5b5bf5;      --brand-2: #8b5cf6;   --brand-soft: #eef0ff;
    --ink: #1b1f36;        --ink-2: #4b5169;     --muted: #8a90a8;
    --bg: #f4f6fc;         --card: #ffffff;      --border: #e6e9f5;
    --success: #16a34a;    --warning: #d97706;   --danger: #e11d48;
    --radius: 16px;        --radius-sm: 10px;
    --shadow-sm: 0 2px 8px rgba(27,31,54,.06);
    --shadow:    0 10px 30px rgba(27,31,54,.08);
    --shadow-lg: 0 24px 60px rgba(27,31,54,.14);
    --font: "PingFang SC", "Microsoft YaHei", "Hiragino Sans GB", -apple-
system,
            BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}
```

后台样式表则定义了侧边栏宽度（--sidebar-w 226px）与顶栏高度（--topbar-h 60px），侧边栏采用深色背景（#161a2e）固定于左侧，右侧为内容区，形成典型的后台管理布局。表格、统计卡、弹窗、分页等组件样式集中在 admin.css 中，便于复用。

6.2 响应式与交互实现

页面通过 <meta name="viewport" content="width=device-width, initial-scale=1"> 声明视口，并在样式表中使用媒体查询与弹性布局实现响应式。手机端会自动收起侧边栏、缩小间距、调整字号，问答输入框固定在页面底部便于输入，保证在桌面、平板与手机浏览器上都能正常使用。

交互层面有几个值得说明的实现：

（1）消息气泡：问答页把每条消息渲染为带头像的气泡，用户消息靠右、助手消息靠左，助手消息下方展示来源标签、相似度、耗时与「依据」（命中的标准问题），并提供「有帮助/没帮助」按钮。

（2）发送中状态：提问后立即显示「正在输入」的三点动画气泡，请求返回后再替换为真实答案，避免用户误以为没有响应；发送期间禁用发送按钮，防止重复提交。

（3）输入框自适应：输入框监听 input 事件，随内容自动增高（上限 150px）；按 Enter 发送、Shift + Enter 换行，符合聊天软件的交互习惯。

（4）猜你想问：从后端获取热门问题渲染为标签，点击后自动填充输入框并发送。

（5）破坏性操作确认：删除会话、清空历史、删除用户等操作都会先弹出确认框。

（6）轻提示：所有操作反馈通过 App.toast 以浮层提示展示，2.4 秒后自动淡出。

```
// 发送问题（chat.js 节选）
async function send() {
    const question = input.value.trim();
    if (!question || state.sending) return;
    state.sending = true; sendBtn.disabled = true;
    input.value = ''; input.style.height = 'auto';
    appendMessage({ role: 'user', content: question });
    showTyping();                                    // 显示「正在输入」气泡
    try {
        const res = await App.api('/api/chat/ask', {
            method: 'POST',
            body: { sessionId: state.sessionId, question: question }
        });
        hideTyping();
        state.sessionId = res.sessionId;
        chatTitle.textContent = res.sessionTitle || '对话';
        appendMessage({ role: 'assistant', content: res.answer, source:
res.source,
                        score: res.score, costMs: res.costMs,
                        matchedQuestion: res.matchedQuestion });
        // 重拉消息，拿到消息 ID 以启用反馈按钮
        const messages = await App.api('/api/chat/messages/' + res.sessionId);
        body.innerHTML = ''; messages.forEach(appendMessage);
    } catch (e) {
        hideTyping();
        appendMessage({ role: 'assistant', content: '抱歉，' + e.message,
source: 'fallback' });
    } finally {
        state.sending = false; sendBtn.disabled = false; input.focus();
    }
}
```

6.3 请求封装与鉴权处理

common.js 中的 App.api 是全局统一的请求封装。它自动携带 Cookie（credentials: same-origin），自动把对象类型的 body 序列化为 JSON 并设置 Content-Type，自动解析统一响应体；更重要的是它统一处理了鉴权失败：当响应状态为 401 时提示「登录已失效」并按当前路径跳转到前台或后台登录页，为 403 时提示「没有访问权限」。对于探测登录态的接口，可传入 silent 选项抑制提示与跳转。

```
async function api(url, options) {
    const opts = Object.assign({ credentials: 'same-origin', headers: {} },
options || {});
    const silent = opts.silent === true; delete opts.silent;
    if (opts.body && typeof opts.body !== 'string') {
```

```
        opts.headers['Content-Type'] = 'application/json';
        opts.body = JSON.stringify(opts.body);
    }
    let res;
    try { res = await fetch(url, opts); }
    catch (e) { throw new Error('网络连接失败，请检查服务是否已启动'); }

    if (res.status === 401) {
        if (!silent) {
            toast('登录已失效，请重新登录', 'error');
            setTimeout(function () {
                location.href = location.pathname.indexOf('/admin') === 0 ?
'/admin/login' : '/login';
            }, 900);
        }
        throw new Error('未登录');
    }
    if (res.status === 403) { if (!silent) toast('没有访问权限', 'error');
throw new Error('无权限'); }

    let data;
    try { data = await res.json(); } catch (e) { throw new Error('服务返回格式
异常'); }
    if (data.code !== 200) throw new Error(data.message || '操作失败');
    return data.data;                                    // 直接返回业务数据
}
```

App 还提供了若干通用工具：esc 对用户内容做 HTML 转义，防止脚本注入；formatTime 与 fromNow 用于时间格式化（绝对时间与相对时间）；currentUser 静默探测当前登录用户；logout 与 bindLogout 处理退出登录；sourceTag 根据答案来源渲染「知识库命中/大模型生成/未命中」标签。后台的 AdminLayout.init 则统一完成管理员身份校验与侧边栏、顶栏的渲染，并提供 renderPager 通用分页组件。

6.4 图表渲染

数据看板的四张图表使用 ECharts 5.5.0（通过 CDN 引入）。dashboard.js 在拿到 DashboardVO 后，分别渲染：近 7 天提问趋势（平滑折线 + 渐变面积）、答案来源占比（环形饼图）、各分类知识条目数量（柱状图）、用户活跃时段分布（24 小时柱状图）。代码中对图表库加载失败做了兜底：若 ECharts 未能从 CDN 加载，则在图表容器中给出文字提示，不影响看板其余内容的展示。窗口尺寸变化时会调用图表实例的 resize 自适应。

```
// 近 7 天提问趋势（dashboard.js 节选）
echarts.init(document.getElementById('chartTrend')).setOption({
    color: PALETTE,
    tooltip: { trigger: 'axis' },
    grid: baseGrid,
```

```
    xAxis: { type: 'category', data: trend.map(r => r.day), boundaryGap:
false, ... },
    yAxis: { type: 'value', minInterval: 1, ... },
    series: [{
        type: 'line', smooth: true, symbolSize: 7,
        data: trend.map(r => r.num),
        areaStyle: { color: new echarts.graphic.LinearGradient(0, 0, 0, 1, [
            { offset: 0, color: 'rgba(91,91,245,.32)' },
            { offset: 1, color: 'rgba(91,91,245,0)' }]) },
        lineStyle: { width: 3 }
    }]
});
// 24 小时活跃分布：补齐 0-23 时缺失的小时，保证 x 轴完整
const hourMap = {};
(data.hourStats || []).forEach(r => { hourMap[Number(r.hour)] =
Number(r.num); });
const hours = [], hourValues = [];
for (let h = 0; h < 24; h++) { hours.push(h + '时'); hourValues.push(hourMap[h]
|| 0); }
```

统计卡与榜单表格则用纯 DOM 拼接渲染。7 个统计卡分别是注册用户、问答会话、提问总数、知识条目、今日提问、知识库命中率与答案满意度；两张榜单是热门知识条目 TOP10（按 hit_count）与用户提问 TOP10（按提问文本聚合）。当没有数据时，表格会显示「暂无数据」占位行，避免空白。

第七章 系统测试

7.1 测试环境与方法

测试在本地开发环境进行，测试环境配置如下表所示。测试方法以黑盒功能测试为主，配合接口测试（通过浏览器开发者工具与接口调试工具验证请求响应）、性能测试思路说明与兼容性测试。

项目	配置
操作系统	Windows 11
JDK	Java 17
构建工具	Maven 3.9
应用框架	Spring Boot 3.2.5（内嵌 Tomcat，端口 8080）
数据库	MySQL 8.0，数据库名 ai_qa，字符集 utf8mb4
浏览器	Chrome 120、Edge 120（另测 Firefox 与手机浏览器）
测试账号	admin/admin123（管理员）、user/123456、demo/123456（普通用户）

表 7-1 测试环境

7.2 功能测试用例

围绕前台与后台功能共设计了 26 条功能测试用例，覆盖注册登录、问答、历史、反馈、知识库/分类/用户管理、配置修改与权限控制等场景。用例与执行结果如下表所示。

编号	测试项	操作步骤	预期结果	实际结果
TC-01	首页展示	打开 http://localhost:8080/	展示站点名称、公告、平台规模数据与热门问题	通过
TC-02	正常注册	填写合法用户名/昵称/密码并提交	提示注册成功并跳转登录页	通过
TC-03	注册-用户名重复	使用已存在的用户名注册	提示「用户名已被注册」	通过
TC-04	注册-密码不一致	两次密码输入不同	提示「两次输入的密码不一致」	通过
TC-05	注册-用户名格式非法	用户名输入不足 4 位或含特殊字符	提示用户名需为 4~20 位字母数字下划线	通过
TC-06	正常登录	使用 user/123456 登录	登录成功并跳转 /chat	通过
TC-07	登录-密码错误	输入错误密码	提示「用户名或密码错误」	通过
TC-08	登录-账号禁用	用被禁用的账号登录	提示「账号已被禁用」	通过
TC-09	未登录访问接口	未登录直接访问 /api/chat/sessions	返回 401 并提示重新登录	通过
TC-10	提问-命中知	提问「图书馆开放时间	返回标准答案，来源为知识	通过

	识库	是什么时候？」	库命中，展示相似度与耗时	
TC-11	提问-关键词变体	提问「图书馆几点开门」	命中同一条知识，展示依据为标准问题	通过
TC-12	提问-未命中	提问与知识库无关的内容	返回兜底话术，来源标记为未命中	通过
TC-13	新建会话	在问答页点击「开启新对话」并提问	创建新会话，标题为问题前 20 字	通过
TC-14	会话续问	在已有会话中继续提问	新消息追加到同一会话，消息数与活跃时间更新	通过
TC-15	答案反馈	点击「有帮助」/「没帮助」	提示感谢反馈，按钮高亮，记录写入数据库	通过
TC-16	猜你想问	查看问答页推荐问题并点击	自动填充问题并发送	通过
TC-17	问答历史	进入 /history 展开会话	展示该会话完整对话内容	通过
TC-18	删除会话	点击某会话的删除按钮并确认	会话及其消息被删除，列表刷新	通过
TC-19	清空历史	点击「清空全部」并确认	全部会话被删除，提示已清空	通过
TC-20	管理员登录	使用 admin/admin123 登录后台	登录成功并进入 /admin 看板	通过
TC-21	普通用户登录后台	用 user 账号登录后台	提示该账号不是管理员，无法登录	通过
TC-22	看板数据	打开 /admin	展示 7 个统计卡、4 张图表与 2 张榜单	通过
TC-23	知识库增删改	新增/编辑/删除/启停知识条目	操作成功，列表刷新，检索快照失效	通过
TC-24	分类删除保护	删除仍有条目的分类	提示该分类下仍有知识条目，无法删除	通过
TC-25	用户禁用保护	禁用当前登录的管理员自身	提示不能禁用当前登录的管理员账号	通过
TC-26	配置修改生效	把问答模式改为 llm 且不填 Key	保存成功；提问时自动降级为本地检索	通过

表 7-2 功能测试用例与结果

说明：TC-11 用于验证相似度算法的泛化能力，即使用户问法与标准问题字面不同，只要关键词命中或二元组重叠足够，仍能命中；TC-26 用于验证失败降级链路，在未配置大模型密钥的情况下，问答功能依然可用。

7.3 接口测试

接口测试通过浏览器开发者工具的 Network 面板与接口调试工具对关键接口进行验证，重点检查请求方法、参数绑定、响应结构与状态码是否符合设计。测试结果如下表所示。

接口	方	测试入参	期望响应	结

	法			果
/api/auth/login	POST	{"username":"user","password":"123456"}	code=200，data 含 token 与用户信息	通过
/api/auth/login	POST	密码错误	code=1003，message=用户名或密码错误	通过
/api/chat/ask	POST	{"sessionId":null,"question":"如何注册账号？"}	code=200，data 含 answer/source/score/sessionId	通过
/api/chat/ask	POST	question 为空	code=400，提示问题内容不能为空	通过
/api/chat/sessions	GET	current=1&size=10	code=200，返回分页会话列表	通过
/api/chat/messages/1	GET	会话属于他人	code=403，没有访问权限	通过
/api/chat/feedback	POST	{"messageId":2,"feedback":1}	code=200，反馈已保存	通过
/api/public/site	GET	无（免登录）	code=200，返回站点名称与公告	通过
/api/admin/stats/dashboard	GET	管理员登录后	code=200，返回 DashboardVO 各字段	通过
/api/admin/knowledge/page	GET	keyword=注册	code=200，返回匹配条目	通过
/api/admin/user/page	GET	role=ADMIN	code=200，仅返回管理员	通过
/api/admin/config/save	PUT	{"configKey":"ai.engine.threshold","configValue":"0.3"}	code=200，参数已保存	通过

表 7-3 接口测试结果

7.4 性能与兼容性测试

性能测试方面，考虑到本项目为课程作品、部署在单机环境，主要采用观测与压测思路结合的方式进行评估：

（1）响应时间观测：通过问答接口返回的 costMs 字段（引擎耗时）与浏览器 Network 面板的耗时统计，在默认演示数据下，本地检索问答的引擎耗时通常在数十毫秒量级，接口整体响应时间在数百毫秒以内。

（2）连接池观测：HikariCP 配置了连接池名称 AiQaHikariPool，可通过日志观察连接获取与归还情况，最大连接数 20 足以支撑单机演示场景。

（3）缓存有效性验证：连续多次提问，观察日志中知识库快照的加载频率，验证在 60 秒内只加载一次，说明快照缓存生效。

（4）分页与上限验证：请求 /api/admin/knowledge/page?size=1000，验证实际返回条数被限制在 100 以内，说明单页上限生效。

兼容性测试方面，分别在 Chrome、Edge、Firefox 以及手机浏览器上打开前台页面，验证布局与交互是否正常；重点检查了响应式断点下侧边栏收起、问答输入框定位与后台表格横向滚动等表现。测试结果显示各主流浏览器均能正常使用。

7.5 测试结论

经过功能、接口、性能与兼容性测试，系统各项功能均按设计正常工作：

（1）26 条功能测试用例全部通过，前台与后台的主要业务流程闭合，异常分支（重复注册、密码错误、账号禁用、未登录、越权、删除保护、降级）均能被正确处理并给出友好提示。

（2）接口的请求方法、参数校验、响应结构与状态码符合设计约定，鉴权与权限控制有效。

（3）问答引擎在知识库命中、关键词变体、未命中三种场景下均表现符合预期，大模型缺失时的降级链路有效。

（4）性能优化手段（快照缓存、分页上限、连接池、统计下推）均得到验证，在演示数据规模下系统响应流畅。

（5）页面在主流浏览器与手机端均可正常使用，响应式布局有效。

总体而言，系统达到了需求分析阶段设定的功能与质量目标，具备可演示、可部署、可维护的基本条件。

第八章 系统部署与运行

8.1 环境准备

运行本系统需要准备以下环境：

（1）JDK 17：安装 JDK 17 并配置 JAVA_HOME，命令行执行 java -version 应显示 17.x。

（2）Maven 3.6 及以上：用于依赖管理与构建打包，命令行执行 mvn -v 验证。

（3）MySQL 8.0：安装并启动 MySQL 服务，准备好具有建库权限的账号（默认使用 root）。

（4）浏览器：推荐 Chrome 90+ 或 Edge 90+，用于访问前台与后台页面。

（5）网络：若使用大模型模式需要能够访问所配置的接口地址；使用本地检索模式时完全离线可用。

8.2 数据库部署

数据库部署只需执行项目自带的初始化脚本 sql/ai_qa.sql。该脚本会先删除同名数据库再重新创建，并建表、建索引、插入演示数据，可重复执行。执行方式有两种：

方式一：在 MySQL 命令行中执行 source 命令，例如：

```
mysql -u root -p
mysql> source d:/Web 大作业/20232653+物联网 2301/源代码/sql/ai_qa.sql;
```

方式二：使用 Navicat、MySQL Workbench 等图形化工具打开该 SQL 文件并整体运行。

脚本执行完成后会自动打印一条校验查询，显示用户数、分类数、知识条目数、会话数、消息数与参数数，便于确认初始化是否成功。正常情况下用户数应为 3、分类数为 6、知识条目数为 27、会话数为 3、消息数为 11、参数数为 7。

8.3 配置修改与启动

启动前需要根据本机环境修改 src/main/resources/application.yml 中的数据源配置，主要是 url 中的主机、端口与数据库名，以及 username 和 password。默认配置如下：

```
spring:
  datasource:
    driver-class-name: com.mysql.cj.jdbc.Driver
    url:
jdbc:mysql://localhost:3306/ai_qa?useUnicode=true&characterEncoding=utf8&

serverTimezone=Asia/Shanghai&useSSL=false&allowPublicKeyRetrieval=true
    username: root
    password: 123456
    hikari:
      pool-name: AiQaHikariPool
      minimum-idle: 5
      maximum-pool-size: 20
      connection-timeout: 30000
      max-lifetime: 1800000
      connection-test-query: SELECT 1
```

启动方式有两种：

方式一（开发调试）：在项目根目录执行 mvn spring-boot:run，等待控制台打印启动成功横幅后，访问 http://localhost:8080/ 即可。

方式二（打包运行）：执行 mvn clean package 生成可执行 jar，再执行 java -jar target/ai-qa-platform.jar 运行。

启动成功后控制台会打印如下提示，明确给出前台与后台的访问地址：

```
================================================
  AI 智能问答网站启动成功
  前台地址: http://localhost:8080/
  后台地址: http://localhost:8080/admin/login
================================================
```

默认账号：管理员 admin / admin123，普通用户 user / 123456 与 demo / 123456。首次使用建议先用管理员账号登录后台，检查知识库与系统配置是否符合需要。

8.4 常见问题排查

现象	可能原因	排查与解决
启动报数据库连接失败	MySQL 未启动或数据源配置错误	确认 MySQL 服务已启动；核对 url、username、password；确认已执行 ai_qa.sql
页面中文乱码	字符集配置不一致	确认数据库为 utf8mb4；application.yml 已配

		置 UTF-8 编码
登录后仍提示未登录	Cookie 未写入或跨域	确认通过 localhost 访问；检查浏览器是否禁用 Cookie
问答总是返回兜底话术	知识库无匹配或阈值过高	检查知识条目是否启用；适当调低 ai.engine.threshold
切换 llm 后仍返回知识库答案	未配置 API Key 或接口不可达	在系统配置中填写正确的 baseUrl 与 apiKey；查看服务端日志的调用失败原因
后台图表不显示	ECharts CDN 未加载	检查网络；看板会给出文字提示，可改用本地 ECharts 文件
端口 8080 被占用	端口冲突	修改 server.port，或结束占用 8080 的进程
修改知识库后答案未更新	快照缓存未失效	正常情况下保存时会自动失效；若异常可等待 60 秒或重启

表 8-1 常见问题排查

第九章 总结与展望

9.1 完成的工作

本项目围绕「智问 · AI 智能问答平台」这一主题，完整地走过了需求分析、系统设计、编码实现、测试与部署的全过程，主要完成的工作包括：

（1）完成了需求分析，梳理出普通用户与管理员两类角色，明确了前台 10 项、后台 7 项功能性需求以及性能、安全、易用性等非功能性需求，并给出了核心用例描述。

（2）完成了系统总体设计，确定了 B/S 多层架构，完成了技术选型，规划了分层结构与工程目录，并绘制了登录、问答、会话等核心流程。

（3）完成了数据库设计，设计了 6 张数据表、17 个索引，编写了可重复执行的初始化脚本，并给出了完整的字段说明与优化策略。

（4）完成了后端开发，实现了统一响应、全局异常处理、JWT 鉴权与权限拦截、公共字段自动填充等公共能力，并按分层完成了认证、问答、知识库、分类、用户、配置、统计等模块。

（5）完成了问答引擎的设计与实现，采用策略模式解耦本地检索与大模型两种回答方式，设计了加权打分公式与字符二元组相似度算法，并实现了内存快照缓存与失败降级。

（6）完成了前端开发，实现了前台 5 个页面与后台 7 个页面，封装了统一的请求与鉴权处理，使用 ECharts 完成了数据看板的可视化。

（7）完成了系统测试，设计了 26 条功能测试用例与接口测试用例，验证了功能、接口、性能与兼容性。

（8）编写了部署与运行说明，整理了常见问题排查表，保证系统可交付、可复现。

9.2 存在的不足

受时间与个人能力所限，系统仍存在一些不足，需要如实说明：

（1）相似度算法较为基础：采用字符二元组与杰卡德相似度，虽然实现简单、无需分词，但语义理解能力有限。对于同义不同词、语序差异较大的问法，可能无法命中或命中的不是最优条目。

（2）大模型模式依赖外部服务：切换 llm 模式后需要可用的接口地址与密钥，且未实现流式输出（打字机效果），回答需要等待完整生成后才能展示。

（3）缺少多轮对话上下文：当前每轮提问独立检索，未把前几轮对话作为上下文参与理解，对于「那周末呢」这类省略主语的追问，只能依靠关键词与相似度，效果不够理想。

（4）反馈闭环不完整：虽然记录了「有帮助/没帮助」评价，但尚未把差评问题自动汇总为待补充知识条目。

（5）权限粒度较粗：目前只区分 ADMIN 与 USER 两种角色，未实现更细粒度的权限控制。

（6）测试以手工为主：缺少自动化单元测试与集成测试，回归成本较高。

9.3 后续改进方向

针对上述不足，后续可以从以下方向改进：

（1）提升检索质量：引入中文分词（如 IK 分词）或词向量模型，用余弦相似度替代杰卡德相似度；或把知识条目向量化后存入向量库，实现语义检索。

（2）支持流式回答：把大模型接口改为流式（SSE），实现逐字输出的打字机效果，提升等待体验。

（3）引入多轮上下文：把会话内最近若干轮对话拼入大模型的 messages，使回答能够理解追问与省略。

（4）完善反馈闭环：把「没帮助」的高频问题自动加入知识库待补充清单，形成运营闭环。

（5）细化权限模型：引入基于角色的访问控制（RBAC），支持更多角色与细粒度权限。

（6）补充自动化测试：为核心算法（TextUtils、打分逻辑）编写单元测试，为接口编写集成测试，保证重构安全。

（7）增强可观测性：接入日志采集与接口耗时统计，便于定位性能瓶颈。

总的来说，本项目完成了预期的全部功能，代码结构清晰、可读性良好，较完整地体现了 Java Web 开发的核心技能与工程思维。更重要的是，通过这个项目，我对「检索 + 生成」的混合问答架构、分层设计与性能优化有了更具体的理解，为后续深入学习打下了基础。

参考文献

[1] VMware Tanzu. Spring Boot Reference Documentation[EB/OL]. https://docs.spring.io/spring-boot/, 2024.

[2] VMware Tanzu. Spring Framework Reference Documentation[EB/OL]. https://docs.spring.io/spring-framework/reference/, 2024.

[3] Baomidou. MyBatis-Plus 官方文档[EB/OL]. https://baomidou.com/, 2024.

[4] MyBatis. MyBatis 3 Documentation[EB/OL]. https://mybatis.org/mybatis-3/, 2024.

[5] Oracle Corporation. MySQL 8.0 Reference Manual[EB/OL]. https://dev.mysql.com/doc/refman/8.0/en/, 2024.

[6] The Thymeleaf Team. Thymeleaf Documentation[EB/OL]. https://www.thymeleaf.org/documentation.html, 2024.

[7] Brett Wooldridge. HikariCP[EB/OL]. https://github.com/brettwooldridge/HikariCP, 2024.

[8] jwtk. Java JWT (JJWT)[EB/OL]. https://github.com/jwtk/jjwt, 2024.

[9] Apache Software Foundation. Apache ECharts 官方文档[EB/OL]. https://echarts.apache.org/zh/index.html, 2024.

[10] M. Jones, J. Bradley, N. Sakimura. JSON Web Token (JWT): RFC 7519[S]. IETF, 2015.

[11] Project Lombok. Project Lombok[EB/OL]. https://projectlombok.org/, 2024.

[12] Eclipse Foundation. Jakarta Servlet Specification 6.0[S]. 2022.

[13] 王珊, 萨师煊. 数据库系统概论(第 5 版)[M]. 北京: 高等教育出版社, 2014.

[14] 周志明. 深入理解 Java 虚拟机(第 3 版)[M]. 北京: 机械工业出版社, 2019.

[15] 孙卫琴. Tomcat 与 Java Web 开发技术详解[M]. 北京: 电子工业出版社, 2019.

附录 A 接口清单

本附录依据各 Controller 的注解如实整理系统全部接口，共 35 个 REST 接口与 12 个页面路由。接口路径均以项目实际代码为准。

所属 Controller	请求路径	方法	说明	权限
AuthController	/api/auth/register	POST	用户注册	公开
AuthController	/api/auth/login	POST	用户登录	公开
AuthController	/api/auth/admin/login	POST	管理员登录（要求 ADMIN）	公开
AuthController	/api/auth/logout	POST	退出登录（清除 Cookie）	公开
AuthController	/api/auth/me	GET	获取当前登录用户信息	登录
AuthController	/api/auth/profile	PUT	修改昵称与邮箱	登录
AuthController	/api/auth/password	PUT	修改密码	登录
ChatController	/api/chat/ask	POST	提问（核心接口）	登录
ChatController	/api/chat/sessions	GET	我的会话列表（分页）	登录
ChatController	/api/chat/messages/{sessionId}	GET	会话内消息列表	登录
ChatController	/api/chat/sessions/{sessionId}	DELETE	删除会话	登录
ChatController	/api/chat/sessions	DELETE	清空我的会话	登录
ChatController	/api/chat/feedback	POST	答案反馈	登录
ChatController	/api/chat/guess	GET	猜你想问（推荐问题）	登录
PublicController	/api/public/site	GET	站点名称与公告	公开
PublicController	/api/public/hot-questions	GET	推荐问题	公开
PublicController	/api/public/overview	GET	平台规模数据	公开
CategoryAdminController	/api/admin/category/list	GET	分类列表	管

			（含条目数）	理员
CategoryAdminController	/api/admin/category/options	GET	分类下拉选项	管理员
CategoryAdminController	/api/admin/category/save	POST	新增/修改分类	管理员
CategoryAdminController	/api/admin/category/{id}	DELETE	删除分类	管理员
ConfigAdminController	/api/admin/config/list	GET	全部系统参数	管理员
ConfigAdminController	/api/admin/config/save	PUT	保存系统参数	管理员
KnowledgeAdminController	/api/admin/knowledge/page	GET	知识条目分页查询	管理员
KnowledgeAdminController	/api/admin/knowledge/save	POST	新增/修改知识条目	管理员
KnowledgeAdminController	/api/admin/knowledge/{id}	DELETE	删除知识条目	管理员
KnowledgeAdminController	/api/admin/knowledge/{id}/status	PUT	启用/停用知识条目	管理员
SessionAdminController	/api/admin/session/page	GET	全部会话分页查询	管理员
SessionAdminController	/api/admin/session/{sessionId}/messages	GET	查看会话消息	管理员
SessionAdminController	/api/admin/session/{sessionId}	DELETE	删除会话	管理员
StatsAdminController	/api/admin/stats/dashboard	GET	看板聚合数据	管理员
UserAdminController	/api/admin/user/page	GET	用户分页查询	管理员
UserAdminController	/api/admin/user/{id}/status	PUT	启用/禁用用户	管理员
UserAdminController	/api/admin/user/{id}/password	PUT	重置用户密码	管理员

UserAdminController	/api/admin/user/{id}	DELETE	删除用户	管理员

表 A-1 REST 接口清单

路径	视图	说明
/	index	前台首页
/login	login	登录页
/register	register	注册页
/chat	chat	智能问答页
/history	history	问答历史页
/admin/login	admin/login	管理员登录页
/admin	admin/dashboard	数据看板
/admin/knowledge	admin/knowledge	知识库管理
/admin/category	admin/category	分类管理
/admin/user	admin/user	用户管理
/admin/message	admin/message	问答记录
/admin/config	admin/config	系统配置

表 A-2 页面路由清单

附录 B 工程文件清单

本附录列出项目的主要源文件，便于对照查阅。其中 Java 源文件 57 个、Thymeleaf 页面 12 个、前端脚本 14 个、样式表 2 个、Mapper XML 1 个，合计 86 个源文件。

类别	文件	说明
构建/配置	pom.xml	Maven 依赖与构建配置（Spring Boot 3.2.5 / Java 17）
构建/配置	application.yml	服务端口、数据源、连接池、MyBatis-Plus 与业务自定义配置
数据库	sql/ai_qa.sql	建库、建表、索引与初始化数据
启动类	AiQaApplication.java	Spring Boot 启动类
common	Result / PageResult / ResultCode	统一响应、分页结果、状态码枚举
common	BusinessException / GlobalExceptionHandler	业务异常与全局异常处理
common	JwtUtil / AuthInterceptor / LoginUser / UserContext	JWT、拦截器、登录用户与线程上下文
config	MybatisPlusConfig	分页插件（单页上限 100）
config	MyMetaObjectHandler	公共时间字段自动填充
config	WebMvcConfig	拦截器注册与路径放行
controller	AuthController / ChatController / PublicController / PageController	前台接口与页面路由
controller.admin	6 个 *AdminController	知识库/分类/用户/会话/统计/配置管理接口
engine	AnswerEngine / AnswerStrategy / AnswerResult	问答引擎、策略接口与结果载体
engine	LocalRetrievalStrategy / LlmStrategy	本地检索策略与大模型策略
entity	6 个实体类	SysUser / QaCategory / QaKnowledge / QaSession / QaMessage / SysConfig
mapper	6 个 Mapper 接口	对应 6 张表的 BaseMapper 及统计查询
mapper	QaKnowledgeMapper.xml	知识条目分页联表查询 SQL
service	6 个 Service	用户/问答/知识库/分类/配置/统计业务服务
dto	6 个入参对象	登录/注册/提问/知识条目/分类/密码表单
vo	3 个出参对象	LoginVO / ChatResponse / DashboardVO
util	PasswordUtil / TextUtils	密码加盐哈希与文本相似度工具
templates	前台 5 个页面	index / login / register / chat / history
templates	后台 7 个页面	admin 下的 login / dashboard / knowledge / category / user / message / config
static/js	前台 6 个脚本	common / index / login / register / chat / history
static/js/admin	后台 8 个脚本	admin-common / login / dashboard / knowledge / category / user / message / config
static/css	2 个样式表	style.css（前台）/ admin.css（后台）

表 B-1 工程文件清单